AUTOMOTIVE DIAGNOSTIC AND ACTIVITY MONITORING SYSTEM

Travis Samson Fernandes	**202107388**
Chetan Naik	**202107334**
Mangesh Phadte	**202107328**
Gaurang Madkaikar	**202107285**

Dissertation submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER ENGINEERING

OF GOA UNIVERSITY

2024 - 2025

COMPUTER ENGINEERING

PADRE CONCEICAO COLLEGE OF ENGINEERING

VERNA GOA – 403722

AUTOMOTIVE DIAGNOSTIC AND ACTIVITY MONITORING SYSTEM

Travis Samson Fernandes	**202107388**
Chetan Naik	**202107334**
Mangesh Phadte	**202107328**
Gaurang Madkaikar	**202107285**

Dissertation submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER ENGINEERING

OF GOA UNIVERSITY

2024 - 2025

COMPUTER ENGINEERING

PADRE CONCEICAO COLLEGE OF ENGINEERING

VERNA GOA – 403722

PADRE CONCEICAO COLLEGE OF ENGINEERING

VERNA GOA – 403722

AUTOMOTIVE DIAGNOSTIC AND ACTIVITY MONITORING (ADAM) SYSTEM

Bona fide record of work done by

Travis Samson Fernandes	**202107388**
Chetan Naik	**202107334**
Mangesh Phadte	**202107285**
Gaurang Madkaikar	**202107328**

Dissertation submitted in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Computer Engineering under Goa University.

2024 - 2025

Approved By:

……...……

Vijaykumar Naik Pawar
Guide

……...…….. ……...……

Dr. Supriya Patil
Head of the Department

Dr. Mahesh Parappagoudar
Principal

Examined By:

Examiner 1: ……....……

(Name / Signature / Date)

Examiner 2: ……....……

(Name / Signature / Date)

PADRE CONCEICAO COLLEGE OF ENGINEERING

VERNA GOA – 403722

STUDENT DECLARATION

We hereby declare that the project report titled **"AUTOMOTIVE DIAGNOSTIC AND ACTIVITY MONITORING SYSTEM"** submitted by us to Padre Conceição College of Engineering, Verna in partial fulfillment of the requirement for the award of the degree of Bachelor of Engineering in Computer Engineering under Goa University is a record of bonafide project work carried out by us under the guidance of Vijaykumar Naik Pawar.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Student Name and Roll No.		**Signature with Date**
1. Travis Samson Fernandes	202107388	
2. Chetan Naik	202107334	
3. Mangesh Phadte	202107328	
4. Gaurang Madkaikar	202107285	

CONTENTS

ACKNOWLEDGMENT i

ABSTRACT ii

LIST OF FIGURES iii

LIST OF TABLES iv

LIST OF ABBREVIATIONS v

1 INTRODUCTION 1

1.1 PURPOSE 1
1.2 PROJECT SCOPE 1
1.3 LITERATURE SURVEY 2
1.4 CHAPTER SUMMARY 13

2 OVERALL DESCRIPTION 14

2.1 PROJECT PERSPECTIVE 14
2.2 PROJECT FUCNCTIONS 15
2.3 USER CLASSES AND CHARACTERISTICS 15
2.4 OPERATING ENVIROMENT 16
2.5 CHAPTER SUMMARY 16

3 REQUIREMENTS 18

3.1 FUCNTION REQUIREMENTS 18
3.2 OTHER NONFUCNTIONAL REQUIREMENTS 18
3.3 HARDWARE REQUIREMENTS 19
3.4 SOFTWARE REQUIREMENTS 19
3.5 CHAPTER SUMMARY 21

4 SYSTEM DESIGN 22

4.1 SYSTEM ARCHITECTURE OVERVIEW 22
4.2 CHAPTER SUMMARY 26

5 IMPLEMENTATION **27**
5.1 INTRODUCTION 27
5.2 GOLANG 32
5.3 POSTGRES DB 36
5.4 FASTAPI 36
5.5 MOBILE APPLICATION 46

6 CONCLUSION **60**

7 BIBLIOGRAPHY **61**

ACKNOWLEDGMENT

We take this opportunity to whole-heartedly thank those who were instrumental in helping us achieve our goals. We extend our heartfelt gratitude to our Principal, **Dr. Mahesh Parappagoudar**, for his unwavering support and encouragement throughout the duration of this project. His visionary leadership and constant motivation have been invaluable to us. We

are equally grateful to our Head of Department, **Dr. Supriya Patil**, for providing us with the necessary resources, facilities, and guidance that played a crucial role in the successful completion of our work. We would like to express our deepest appreciation to our project guide, **Vijaykumar Naik Pawar**, whose insightful suggestions, constructive feedback, and dedicated support were instrumental in shaping this project. Their expertise and constant encouragement inspired us to push our limits and achieve the desired outcomes. We would also like to thank

the staff of the **Department of Computer Engineering** for their cooperation and support. We also wish to acknowledge the valuable contributions of the faculty members, whose knowledge and guidance have enriched our understanding throughout the course of this endeavour. Additionally, we are thankful to our peers and friends for their encouragement, collaboration, and constructive feedback, which motivated us to refine our work and strive for excellence.

ABSTRACT

The Automotive Diagnostic and Activity Monitoring (ADAM) System is a ground-breaking initiative aimed at revolutionizing the way vehicles are operated, monitored, and maintained. Leveraging advanced technologies, ADAM seeks to enhance road safety, streamline user convenience, and offer robust forensic support. By collecting and analysing critical vehicular data prior to accidents, the system proactively identifies and addresses potential risks, reducing the likelihood of road incidents. This preventive mechanism not only safeguards drivers and passengers but also contributes to building safer roadways through timely and effective risk mitigation strategies.

One of the standout features of the ADAM System lies in its utility for forensic investigations. By automating the monitoring and recording of vehicle activities, the system becomes an indispensable tool for law enforcement agencies and insurance companies. It facilitates the reconstruction of accident scenarios with high accuracy, aiding in the determination of liability and the resolution of disputes. This functionality ensures greater transparency and fairness in investigations, accelerating claims processing and fostering trust among stakeholders.

In addition to its safety and forensic applications, the ADAM System empowers vehicle owners by providing advanced diagnostic tools to detect and address mechanical issues with precision. By equipping users with reliable insights into their vehicle's condition, it enables informed maintenance decisions, protects them from potential scams by unscrupulous mechanics, and minimizes repair costs. These capabilities ensure that vehicles remain in peak operating condition, reducing downtime and extending their lifespan.

The ADAM System represents a paradigm shift in vehicle operation by seamlessly integrating data collection, real-time analysis, and system automation. Its intelligent features not only enhance vehicle security but also simplify routine processes, making it an indispensable tool for modern-day vehicle owners. By prioritizing safety, convenience, and efficiency, ADAM aspires to redefine the relationship between drivers and their vehicles. Through its innovative and user-focused approach, the system aims to create a driving experience that is safer, smarter, and more intuitive, ultimately contributing to a more connected and secure transportation ecosystem.

LIST OF FIGURES

Figure	Name	Page No.
Fig. 4.1	System Design Overview	(22)
Fig. 5.1	Automotive Diagnostic And Activity Monitoring	(27)
Fig. 5.2	Flow Diagram Of Raspberry Pi	(28)
Fig. 5.3	Predicted Engine Condition Distribution (Good vs. Needs Attention)	(36)
Fig. 5.4	Comprehensive Flowchart of the Predictive Maintenance Router Diagnostic Process	(43)
Fig. 5.5	Dashboard of the ADAM mobile application	(46)
Fig. 5.6	The dashboard includes direct navigation to four essential module	(46)
Fig. 5.7	The bottom tab of the ADAM mobile application provides quick access to key features	(47)

LIST OF TABLES

Table	Name	Page No.
Fig. 1.1	Literature Review Summary	(5)
Fig. 4.1	Modules and their Functional Descriptions	(29)
Fig. 4.2	Device to GPIO Pin Connections on Raspberry Pi	(29)

LIST OF ABBREVIATIONS

Name	Description
ADAM:	Automotive Diagnostic and Activity Monitoring
OBD:	On-Board Diagnostics Port
DTC:	Diagnostic Trouble Codes
ECU:	Engine Control Unit
RTO:	Regional Transport Office
GPS:	Global Positioning System
IAT:	Intake Air Temperature
MAF:	Mass Air Flow

INTRODUCTION

The Automotive Diagnostic and Activity Monitoring (ADAM) System is a pioneering project designed to enhance vehicle safety, improve maintenance efficiency, and support forensic investigations. By leveraging technologies such as Raspberry Pi, machine learning, and mobile app integration, ADAM provides real-time diagnostics, driving insights, and data collection capabilities. This comprehensive system aims to empower users, promote safer roads, and establish transparency in automotive operations.

1.1 PURPOSE

The Automotive Diagnostic and Activity Monitoring (ADAM) System is designed to redefine vehicle safety, user convenience, and forensic capabilities through innovative technology. By providing real-time diagnostics and driving insights, ADAM aims to promote safer roads, reduce risks, and ensure efficient vehicle maintenance. The system is also equipped to collect and analyze forensic data for post-accident investigations, aiding in uncovering critical information for law enforcement, insurance companies, and vehicle owners.

ADAM empowers users with the ability to understand and monitor their vehicles, enabling informed decisions about maintenance and repairs while safeguarding against fraudulent practices in the automotive repair industry. This dual focus on proactive prevention and forensic analysis establishes the ADAM System as a transformative tool in modern automotive management.

1.2 PROJECT SCOPE

The ADAM System leverages a blend of hardware and software technologies, including Raspberry Pi, machine learning algorithms, and mobile app integration, to deliver comprehensive solutions for vehicle monitoring and diagnostics. The scope of the project encompasses:

1.2.1 REAL-TIME ENGINE DIAGNOSTIC AND ALERTS

The system continuously retrieves critical data from the vehicle's OBD-II port, such as engine performance metrics, error codes, and fuel efficiency. Real-time analysis of this data ensures that users are promptly alerted to potential issues, enabling timely interventions to prevent severe damage or unsafe conditions.

1.2.2 DRIVING PATTER ANALYSIS FOR INSURANCE AND GOVERNMENT USE

By analyzing driving behaviors such as speed, acceleration, and braking patterns, ADAM provides valuable insights for insurance companies to customize premiums based on driver performance. Additionally, these analytics can support government initiatives in traffic safety and urban planning.

1.2.3 ENHANCE VEHICLE SAFETY AND ACCIDENT ANALYSIS

Through advanced data collection and secure storage, ADAM functions as a black box for vehicles. It captures pre-accident data, which can be invaluable for forensic investigations, helping to establish circumstances leading to an incident and enhancing overall road safety.

By integrating cutting-edge technologies, the ADAM System aims to revolutionize how vehicles are operated, maintained, and monitored, ensuring a safer and more transparent experience for all stakeholders.

1.3 LITERATURE SURVEY

1.3.1 OVERVIEW

The development of the ADAM System draws on extensive research into OBD-II standards, vehicle diagnostics, and data security. The goal was to identify methodologies and technologies that ensure precise data collection, effective analysis, and secure handling. Studies emphasize the role of standardized protocols like PIDs and CAN in creating robust systems for automotive diagnostics and monitoring.

A thorough review of relevant research papers has refined the project's approach, combining proven methodologies with innovative applications. Key areas include vehicle data collection techniques, the integration of embedded systems like Raspberry Pi, and applications of machine learning for real-time diagnostics and forensic analysis.

1.3.2 Key Research Insights

1. **OBD-II Protocols and Standards**: Le Nguyen et al. (2024) highlight the importance of using standardized Parameter Identifications (PIDs) and CAN protocols for accurate data retrieval and diagnostics. Their research demonstrated the integration of e-black boxes with GPS and cameras to collect critical data for accident investigations, vehicle performance analysis, and driving behavior monitoring.

2. **Embedded System Integration**: Sawant & Mane (2024) explored the use of MSCAN modules with KEAZ128 microcontrollers for vehicle diagnostics. Their system focused on detecting and monitoring malfunctions using wireless communication between the OBD-II con-

nector and external systems. The study's experiments underscored the reliability of integrating microcontrollers and suggested future expansions to include additional diagnostic capabilities.

3. **Data Security and Analysis**: Multiple studies underscore the importance of securing vehicle data using encryption techniques during storage and transmission. This ensures the integrity of sensitive information and prevents unauthorized access, which is vital for both user trust and forensic validity.

1.3.3 Applications of Research

The literature review significantly influenced the ADAM System's design by emphasizing the importance of standardized OBD-II protocols, embedded system architectures, and data encryption to ensure reliable and secure vehicle diagnostics. Research highlights the versatility of OBD data not only for reading engine parameters and fault codes but also for offering real-time insights into driving behavior, emissions, and fuel efficiency. Recent studies have demonstrated the use of OBD-derived metrics combined with smartphone sensors and machine learning models to classify driving patterns, detect aggressive driving, and forecast emissions. Additionally, machine learning algorithms such as Support Vector Machines, AdaBoost, and Random Forest have been successfully applied to categorize driver behavior based solely on OBD data. Physics-informed machine learning models have also shown significant improvements in predicting vehicle emissions using real-time OBD parameters. These findings validate the ADAM System's use of OBD-II data for advanced analytics, predictive maintenance, and forensic diagnostics.

Alongside diagnostics, the survey revealed important advancements in vehicle cybersecurity and lightweight embedded encryption that shaped the ADAM System's secure architecture. As the Controller Area Network (CAN) lacks native encryption and authentication, it is vulnerable to message injection, replay, and denial-of-service attacks. In response, researchers have proposed security measures such as network segmentation, intrusion detection systems, and lightweight authentication protocols. Techniques like stream cipher-based encryption for CAN messages, diagnostic firewalls at the OBD-II port, and clock-based intrusion detection systems have proven highly effective. Recent models using generative adversarial networks, LSTM, and autoencoders have achieved over 99 percent detection rates for common attack types. These developments directly influenced the ADAM System's secure design, incorporating encrypted data transmission, segmented data handling, and anomaly detection to support reliable, forensic-grade diagnostics and proactive protection against cyber threats.

Table 1.1: Literature Review Summary

PAPER TITLE	**DESCRIPTION**	**EXPERIMENT/ RESULTS**	**FUTURE WORK**
The Design and Implementation of New Vehicle Black Box Using The OBD Information Duy Le Nguyen, Myung-Eui Lee, Artem Lensky (IEEE Xplore)	The Paper discusses E-Black Box that is a combination of a black box and an event-driven recorder that communicates with a vehicle's ECU module through the OBD-II port to collect vehicle status information. It gathers data from various sources, including the OBD-II port, a camera, GPS, and a 9-degree-of-freedom inertial measurement unit (IMU). This data is logged in real-time to a Secure Digital (SD) card at intervals of 100 milliseconds. The recording process stops when the vehicle stops or an accident occurs and if the memory becomes full, the oldest data is automatically overwritten to ensure continuous operation. The E-Black Box comes with two simulation modes: an online simulation mode that runs in real-time when the vehicle is in operation and an offline simulation mode for debugging and analysis of recorded data. It also comes with smartphone connectivity via Bluetooth, which allows it to send the vehicle's status and location in case of an accident. It can also search for nearby service centers or hospitals based on its status and location.	Accident Investigation, Vehicle Performance, Driver Behavior Analysis	Road tracking and obstacle detection algorithms

Design and Development of On-Board Diagnostic (OBD) Device for Cars Pooja Rajendra Sawant, Yashwant B Mane (IEEE Xplore)	This research paper discusses the design and development of an On-Board Diagnostic (OBD) system for cars, based on OBD-II standards established by the Society of Automotive Engineers (SAE). OBD-II standards have been mandatory for all cars sold after 1996. The system provides real-time information about a vehicle, such as engine speed, coolant temperature, pressure, and Diagnostic Trouble Codes (DTCs). DTCs are five-character codes that have a predefined structure, helping users know the actual status of their vehicles in real time and diagnose malfunctions. Codes can be generic or manufacturer-specific. OBD systems communicate with the CAN bus to enable the vehicle's ECU to communicate with the OBD device through the ISO 15765 standard. The device, according to the paper, employs a KEAZ128 microcontroller, an automotive-grade controller, to interpret the OBD-II protocol and act as a bridge between the vehicle and the user's computer.	Wireless communication between OBD connector and PC using Bluetooth, user-friendly GUI	Adding more diagnostic features and data into the device database

Design and Implementation of a Wireless OBD II Fleet Management System Reza Malekian, Ntefeng Ruth Moloisane, Lakshmi Nair, BT(Sunil) Maharaj, Uche A.K. Chude-Okonkwo (IEEE Xplore)	This paper introduces a wireless system for fleet management, monitoring vehicle speed and fuel consumption, and computing distance by using an OBD-II module along with GPS data. The system uses the ELM327 IC for decoding OBD-II signaling protocols and communicates using AT commands. The OBD-II protocols implemented are ISO 15765 (CAN), ISO 9141-2, and ISO 14230-4. In this system, speed is coded as ”0D” in hexadecimal, and MAF is ”10.” The fuel is calculated based on the measured MAF, and distance is calculated based on the measured speed. This information is logged along with the GPS data simultaneously. All logged data is transmitted to a remote server via a Carambola2 WiFi module, where it is stored in an SQL database. The analysis of the data is made through a graphical interface. The system was tested on a BMW 125i and an OBD-II emulator. The measurements of distance had an error margin of about 5%, mainly due to a 1-second data sampling interval.	Tested with OBD II emulator and a real vehicle. Able to accurately measure speed, fuel consumption, and distance with approximately 5% error over short and long distances. Communication range is 900 meters for WiFi.	Proposed using GSM instead of WiFi for wider range, adding a backup battery to maintain wireless communication when the vehicle is off, and improving initialization time.

Machine Learning Based Automatic Diagnosis in Mobile Communication Networks Kuo-Ming Chen, Tsung-Hui Chang, Kai-Cheng Wang, Ta-Sung Lee (IEEE Xplore)	This paper reports on an ML-based algorithm for automatic fault diagnosis in mobile communication networks. The scheme utilizes SONs, removing the need for expensive hardware by making the networks self-configurable, self-organizing, and self-healing. The presented methodology combines a Softmax NN and SVM models with the aim of improving diagnosis accuracy. It further uses KPIs as well as performance management counters (PMCs) that are used in feature extraction. The combined output from the Softmax NN along with SVM carries out multiclass classification while diagnosing network faults accordingly. Thus the ML-based algorithms are of unsupervised type because they learn the training datasets directly and thereby are malleable in real application scenarios. This dual capability addresses both single and multi-fault scenarios, making the system robust enough for modern mobile networks as an efficient alternative to fault-detection methods.	Achieves high accuracy in both single and multi-fault detection in LTE networks	Explore other models and enhance multi-fault detection.

Vehicular Mobile Commerce Authors [Not provided in excerpt] (IEEE Xplore)	Wi-Fi-enabled vehicles enables m-commerce applications, like entertainment and diagnostics. Significant technical, safety, and financial challenges remain	Highlights real-world prototypes, such as Ford's Lincoln and an Atlanta traffic data project. These illustrate potential but underscore connectivity and safety issues.	Calls for solutions to improve stable computing tech in high-speed connectivity, security, privacy, and cost efficiency to make vehicular m-commerce practical and safe.
Automotive Internal-Combustion-Engine Fault Detection and Classification Using Artificial Neural Network Techniques Ryan Ahmed, Mohammed El Sayed, S. Andrew Gadsden, Jimi Tjong, Saeid Habibi (IEEE Xplore)	This research examines the use of artificial neural networks for the detection and classification of faults in internal combustion engines. The paper has trained such ANNs based on the vibration data measurements in the crank angle domain and with the potential to set up a consistent diagnostic system for dealerships and assembly plants, reducing the cost of manufacturers' warranty. Methodologies in the paper involve feed-forward multilayer perceptron ANNs, with the appeal for these learning, adapting, noise rejecting, and handling nonlinear relations. Training algorithms used here include backpropagation, Levenberg-Marquardt, quasi-Newton, extended Kalman filter, and the newest variable structure filter introduced as a smooth variable structure filter. The variable structure filter is notable because of stability, robustness, and the good response to the uncertainties.	SVSF-ANN shows high accuracy and reliability for fault classification.	Expand to real-time applications and add more engine parameters.

Automotive Internal-Combustion-Engine Fault Detection and Classification Using Artificial Neural Network Techniques	The research benefits the detection process of faults by avoiding special location models of fault. The test bench employed a semi-anechoic chamber with a four-stroke eight-cylinder engine. Triaxial accelerometers were used to collect data of vibrations and then transform this data into the crank angle domain by the help of a cylinder identification sensor. The tested faults included missing bearings, piston chirps, chain tensioners, etc., each of which possesses some specific vibration patterns.	SVSF-ANN shows high accuracy and reliability for fault classification.	Expand to real-time applications and add more engine parameters.
Deep Learning Model Based CO2 Emissions Prediction Using Vehicle Telematics Sensors Data Authors [Not provided in excerpt] (IEEE Xplore)	This paper presents an LSTM model that predicts CO2 emissions from OBD-II sensor data in real-time. The model is designed to handle noisy data and improve time-series prediction accuracy. The study demonstrates the model's effectiveness in predicting emissions and suggests its potential for broader deployment in real-time monitoring systems.	Outperforms other methods in handling noisy data and time-series prediction accuracy.	Use diverse datasets for broader deployment in real-time monitoring systems.

OBD SecureAlert: An Anomaly Detection System for Vehicles Sandeep Nair Narayanan, Sudip Mittal, Anupam Joshi (IEEE Xplore)	This paper presents OBD-SecureAlert, a data-driven anomaly detection system for cars designed to identify abnormal behaviors using CAN bus data. The CAN bus is an internal communication network found in vehicles that connects ECUs, sensors, and actuators. Since most CAN buses lack built-in security measures, they are open to cyberattacks that can compromise the safety of automobiles. OBD-SecureAlert eliminates this vulnerability by utilizing a Hidden Markov Model (HMM) for anomaly monitoring and detection against deviations from normal vehicle behavior. This system emphasizes detection instead of prevention, thereby creating an applicative solution to identify possible threats. It collects data from the CAN bus, which carries the values of speed and RPM sensors in the vehicle. After being trained, it analyzes the incoming data using a sliding window and flags data sets based on predefined threshold limits as anomalies or safety/cybersecurity threats. They simulate attack scenarios, like injecting anomalous data into the CAN bus, to detect anomalies.	Effectively uses a Hidden Markov Model (HMM) to detect anomalies in vehicle behavior by analyzing CAN bus data, successfully identifying threats like malicious data injections in real-time.	Integrating with machine learning models like LSTM or Transformer-based architectures for anomaly detection or real-time threat mitigation against early signs of wear or failure.

Assessing the Impact of Driving Behavior on Instantaneous Fuel Consumption Javier E. Meseguer, Carlos T. Calafate, Juan Carlos Cano, Pietro Manzoni (IEEE Xplore)	This paper presents DrivingStyles, a platform designed to analyze and improve driving habits for reducing fuel consumption and greenhouse gas emissions. The system uses real-time data from vehicle Electronic Control Units (ECUs) through OBD-II Bluetooth interfaces and smartphone technology to evaluate driving behavior and fuel efficiency. The platform employs data mining and neural networks to classify driving styles based on speed, acceleration, and engine RPM. The system architecture includes an Android app, OBD-II interface, and web-based data center that collects and analyzes vehicle data. Study findings show driving style significantly affects fuel usage and emissions, with efficient driving practices achieving 15-20% fuel savings compared to aggressive driving behaviors.	An efficient driving style can drastically reduce fuel consumption and reduce greenhouse gas emissions, with an estimated saving of 15–20%. The DrivingStyles platform, with smartphone and vehicle data, is one of the practical tools to help raise awareness and improve driving styles for better energy efficiency.	Long Short-Term Memory (LSTM) networks or Transformer models to better capture temporal driving patterns and develop individualized driving style profiles based on user-specific data to provide tailored recommendations.

Exploring Fuzzy Logic and Random Forest for Car Drivers' Fuel Consumption Estimation in IoT-Enabled Serious Games Rana Massoud, Francesco Bellotti, Riccardo Berta, Alessandro De Gloria, Stefan Poslad (IEEE Xplore)	This paper explores Fuzzy Logic (FL) and Random Forest (RF) algorithms to estimate vehicle fuel consumption for developing an IoT-enabled driving game that provides real-time feedback to promote efficient driving habits using the enviroCar database. The research uses real-time vehicle data collected through OBD-II interfaces, focusing on throttle position sensor (TPS), RPM, vehicle speed, and fuel consumption across various car models and driving conditions. FL uses "if-then" rules for understandable feedback, while RF applies machine learning for high-accuracy predictions. Results show RF outperforms FL with higher R^2 (0.896 vs. 0.65) and lower MSE (1.506 vs. 4.745), identifying speed and RPM as critical predictors. Combining FL's intuitive coaching with RF's precise estimates enhances IoT-enabled driving games and scales to other real-time IoT applications.	Fuzzy Logic (FL) provides interpretable feedback for coaching, while Random Forest (RF) delivers superior predictive accuracy for fuel consumption modeling. The combination of both models benefits from their strengths and makes them ideal for IoT-enabled driving games and applicable to other fields requiring real-time user performance assessment.	Benchmark the performance of RF and FL against newer algorithms, such as Gradient Boosting (e.g., XGBoost) or neural networks, to validate the choice of models.

1.4 CHAPTER SUMMARY

1. The introduction presents a smart automotive diagnostic and activity monitoring solution that enhances safety, maintenance efficiency, and forensic investigation using Raspberry Pi, machine learning, and mobile integration.

2. The project scope includes real-time engine diagnostics via the OBD-II interface, driving behavior analysis for insurance and traffic safety, and secure black-box-like accident data storage for forensic support.

3. The literature survey highlights the importance of standardized OBD-II protocols, embedded system integration with microcontrollers, and data encryption to ensure accurate diagnostics and secure data handling.

4. By leveraging standardized diagnostic protocols it ensures compatibility with various vehicle systems, securing wireless communication channels, and maintaining reliable data collection in real-time scenarios.

5. The purpose of the ADAM System is to empower users with informed vehicle insights, promote safer driving, assist in post-accident investigations, and establish transparency in automotive diagnostics and maintenance.

OVERALL DESCRIPTION

This chapter delves into the Automotive Diagnostic and Activity Monitoring (ADAM) System, providing an in-depth exploration of its innovative design, core functions, and user-centric features. It outlines the project's integration of advanced hardware and software components to offer comprehensive solutions in vehicle diagnostics, maintenance, and forensic analysis.

2.1 PROJECT PERSPECTIVE

The **Automotive Diagnostic and Activity Monitoring (ADAM) System** is an innovative, standalone automotive diagnostic platform designed to supersede conventional vehicle diagnostic tools with an advanced, integrated ecosystem. It establishes a direct interface with a vehicle's **Engine Control Unit (ECU)** via standard **OBD-II protocols**, enabling continuous real-time data acquisition and analysis. This system provides valuable insights to **vehicle owners, insurers, fleet operators, and government authorities**, enhancing road safety, maintenance scheduling, emissions compliance, and operational transparency.

From a hardware perspective, the ADAM System comprises a **Raspberry Pi 5 (4GB RAM)** as its central processing unit, interfaced with essential automotive-grade sensors such as the **Neo-6M GPS module** for location tracking, the **SW-420 vibration sensor** for impact and abnormal motion detection, and the **MPU-6050 accelerometer and gyroscope** module for capturing dynamic vehicle behavior. Power is supplied through a **6000mAh 11.1V 3S Li-ion battery**, ensuring reliable, continuous operation independent of the vehicle's primary power source.

On the software side, the system architecture is built around a **Golang server using the Fiber framework**, which manages data ingestion and storage via a **PostgreSQL database**. Complementing this, a **FastAPI-based Python service** handles advanced analytics by executing machine learning models that classify engine health conditions, detect emissions anomalies, and generate predictive maintenance schedules. These models process both real-time and historical telemetry to deliver actionable diagnostics, anticipate component failures, and support compliance with environmental standards.

For user interaction, ADAM features a **React Native mobile application built on the Expo framework**, providing an intuitive interface where users can monitor diagnostic data, receive health alerts, track vehicle performance metrics, and access predictive maintenance recommendations. This cohesive integration of hardware, software, intelligent analytics, and user interface establishes the ADAM System as a scalable, reliable, and forward-compatible solution for modern automotive diagnostics and fleet management.

2.2 PROJECT FUCNCTIONS

1. **Black Box Data Collection:** The system records key parameters, including RPM, speed, throttle position, engine temperature, and error codes. This data is stored securely and can be analyzed to understand vehicle performance and pre-accident conditions. By functioning as a black box, ADAM serves as a reliable source of information for investigations, aiding in uncovering critical details about vehicle operations prior to incidents.
2. **Mobile App Visualizations:** Users can access real-time vehicle statistics, diagnostics, and alerts through an intuitive mobile application. The app also provides detailed visualizations and suggestions for maintenance, ensuring users are informed and empowered to take necessary actions. Additionally, the app integrates with cloud services to enable historical data access, trend analysis, and remote monitoring.
3. **Forensic Data Sharing:** ADAM facilitates secure export of vehicle data in CSV format for use by insurance companies and law enforcement agencies. This feature supports claim verification and accident investigations, providing transparency and accountability. Advanced encryption ensures that data remains confidential during transmission, preserving the integrity of sensitive information.

2.3 USER CLASSES AND CHARACTERISTICS

1. **Vehicle Owners:**
   - Monitor real-time vehicle health and performance metrics.
   - Receive proactive alerts for maintenance and potential issues.
   - Access diagnostic tools to understand and resolve problems effectively.
   - Benefit from enhanced transparency in repair processes, reducing the risk of being misled by mechanics.
2. **Insurance Agents:**
   - Utilize driving data to verify claims and assess driver behavior.
   - Access pre-accident data to establish liability and streamline claim processing.
   - Leverage aggregated data insights for policy adjustments and risk assessment.
3. **Government Bodies:**
   - Analyze anonymized driving habits to inform traffic safety policies.
   - Use aggregated data to identify patterns and improve urban planning.
   - Incorporate ADAM's analytics into broader smart city initiatives to optimize transportation networks.

2.4 OPERATING ENVIROMENT

1. **Hardware Components:**

   - **Raspberry Pi 5:** Serves as the core processing unit for data collection and analysis, capable of handling high volumes of real-time data efficiently.
   - **OBD-II Reader:** Interfaces with the vehicle's ECU to retrieve critical diagnostic data. It supports multiple protocols for compatibility with diverse vehicle models.
   - **GPS Module:** Tracks vehicle location and assists in route analysis and forensic investigations, providing geographic context to vehicle data.
   - **Storage Devices:** Ensures secure and encrypted storage of vehicle data for analysis and sharing. Options include SD cards and external drives for scalability.

2. **Software Components:**

   - **Raspberry Pi OS:** The operating system managing hardware functions and software execution, optimized for real-time processing tasks.
   - **Python Libraries:** TensorFlow for predictive analytics, Flask for application interfacing, and encryption libraries for secure data handling. Libraries for GPS data processing and CAN protocol decoding are also integrated.
   - **Mobile Application:** Provides an interface for end-users to access vehicle data, visualize statistics, and receive alerts. Features include cloud synchronization, customizable dashboards, and multi-user support for fleet management.

2.5 CHAPTER SUMMARY

1. The chapter provides an in-depth description of the System architecture, pointing out its mixture of advanced hardware and intelligent software to deliver automotive diagnostics, monitoring, and forensic capabilities

2. The system replaces traditional diagnostic tools by using real-time OBD-II communication, GPS tracking, and data logging on a Raspberry Pi, with analysis via a FastAPI server and results displayed on a mobile app.

3. Key functionalities include black box-style data logging, a feature-rich mobile app for real-time diagnostics, secure forensic data sharing, and support for multiple user roles such as vehicle owners, insurers, and investigators.

4. Distinct user classes, such as individual owners, insurance companies, and government agencies, interact with the system via tailored interfaces to benefit from live monitoring, maintenance, historical data analysis, traffic safety, and forensic investigation.

5. The operating environment integrates robust hardware like Raspberry Pi 5, OBD-II reader, GPS module, and multiple sensors with efficient software technologies including Python, FastAPI, OpenAI, and TensorFlow libraries for secure and scalable deployment.

6. The ADAM system offers a unified platform that ensures real-time data collection, predictive maintenance, secure data transmission, and cloud integration, making it a forward-compatible solution for smart vehicle diagnostics and activity monitoring.

REQUIREMENTS

In this chapter, we will learn about different types of Functional and Non-Functional requirements as well as Hardware and Software requirements, which ensure compatibility, aiding smooth project execution by setting clear expectations and guidelines for development and performance.

3.1 FUCNTION REQUIREMENTS

Functional requirements define the specific operations and functionalities the ADAM System must perform under various conditions. These include:

1. **Real-Time Data Collection:** ADAM continuously retrieves live vehicle data (e.g., RPM, speed, Diagnostic Trouble Codes) from the OBD-II port and ensures reliable, uninterrupted transmission to the Raspberry Pi, even under varying vehicle conditions.
2. **Black Box Functionality:** Acts as a digital event data recorder, persistently storing sensor data during critical events like hard braking or collisions to support forensic analysis and accidents.
3. **Mobile Application Features:** Provides daily logs in interactive graphs, delivers advanced analysis using machine learning algorithms, and features a chatbot interface for users to ask queries and receive instant insights.
4. **Forensic Data Sharing:** Enables controlled sharing of diagnostic data with external parties, ensuring user-defined access and compliance with privacy standards.

3.2 OTHER NONFUCNTIONAL REQUIREMENTS

Nonfunctional requirements describe the system's expected performance and quality attributes. These include:

1. **Reliability:** Ensures high reliability by maintaining continuous uptime, implementing a buffer to securely cache data during connectivity interruptions, and utilizing failover mechanisms to guarantee uninterrupted data collection and secure backup until the data is successfully posted to the database
2. **Security:** security with strong authentication and strict authorization, allowing only verified users appropriate access.

3. **Performance:** The system processes and visualizes vehicle sensor data in real time, ensuring low latency and responsiveness, especially during critical events.

4. **Maintainability:** A modular architecture allows for independent updates to components like the UI, GPS module or other sensors. Well-maintained documentation enables efficient development, debugging, and onboarding.

5. **Scalability:** It supports various vehicle types and OBD-II protocols, allowing the addition of new sensors and integrations with external platforms (e.g., insurance APIs, fleet systems) for enterprise-level expansion.

3.3 HARDWARE REQUIREMENTS

The hardware required to develop and deploy the ADAM System includes:

1. **Raspberry Pi 5:** The Raspberry Pi 5 serves as the central unit for data collection, AI processing, and interfacing with mobile/cloud services, offering high performance and peripheral support.

2. **OBD-II Reader:** Connects to the vehicle's ECU using protocols like CAN to read critical diagnostics data such as RPM, speed, engine temperature, throttle position, etc.

3. **GPS Module:** Provides location tracking for route analysis, theft monitoring, and event tagging to support forensic and insurance purposes.

4. **Storage Devices:** High-endurance SD card is used for scalable storage of large volumes of real-time and historical data.

5. **Power Supply:** Ensures uninterrupted operation using vehicle converters and a relayed battery pack, handling power fluctuations and downtime.

6. **Vibration Module:** Used to detect and transmit vibration or shock occurrences using a spring-based switch, providing a digital output when vibration is present.

7. **Gyroscope and Acceleration:** Combines a 3-axis accelerometer and 3-axis gyroscope to measure movement, orientation, and sudden impacts, enabling accurate motion and accident detection.

3.4 SOFTWARE REQUIREMENTS

To power the ADAM system effectively, a robust software stack is used, covering real-time embedded processing, user interfacing, secure data transmission, and cloud integration.

1. **Raspberry Pi OS:** It is the base system running on the Pi. Chosen for its compatibility with ARM architecture and extensive support for Python libraries and hardware integration, such as GPIO, OBD-II, , etc.

2. **Python:** Primary language for system logic and real-time data handling, chosen for its simplicity and vast ecosystem.

3. **Google Colab** It is a free, cloud-based platform provided by Google that allows users to write and execute Python code in a collaborative environment. It provides access to GPUs, making it popular for machine learning tasks.

4. **TensorFlow/Pytorch/scikit-learn Libraries:** Used for machine learning tasks like building models that detect anomalies or predict component failures based on historical data.

5. **FastAPI** Provides a lightweight web framework to build REST APIs connecting to PostgreSQL and enabling advanced analytics in Python.

6. **PostgreSQL** It is a powerful, open-source relational database known for its reliability, extensibility, ACID compliance, and advanced features that support complex data management and high-performance applications.

7. **Golang:** Uses the Fiber framework to build a high-performance web server, offering an Express-inspired API for rapid development, efficient routing, and minimal resource usage, making it well-suited for creating RESTful APIs and handling concurrent requests with ease

8. **React Native with Expo:** Enables rapid development of cross-platform Android and iOS apps from a single codebase, offering streamlined workflows, easy access to native APIs, and simplified testing and deployment.

9. **Docker:** Containers package the application and its dependencies into isolated, reproducible units, ensuring consistent operation across different systems without requiring cloud integration.

10. **Visual Studio Code:** Visual Studio Code (VS Code) is a free and open-source code editor developed by Microsoft. It supports various programming languages and features a customizable interface and a rich extension ecosystem.

11. **Supabase** It is used for authenticating email, social, and passwordless logins, as well as OAuth, and for storing personal data, functioning like a document store with secure, user-managed access and integrated authorization controls

3.5 CHAPTER SUMMARY

1. The ADAM system requires real-time data collection via the OBD-II port, with secure transmission to a Raspberry Pi for continuous vehicle monitoring.
2. Black box functionality ensures persistent storage of critical vehicle data before and during the incidents for forensic analysis and driver behavior evaluation.
3. A mobile application provides real-time vehicle status, graphical performance metrics, and machine learning analysis
4. Driving behavior is analyzed using machine learning to offer safe driving tips and insights for insurance and regulatory purposes.
5. Nonfunctional requirements include high system reliability, real-time responsiveness, modular maintainability, and scalability for different vehicles and future integrations.
6. Hardware components include Raspberry Pi 5, OBD-II reader, GPS module, encrypted storage (SD/SSD), reliable power supply, MPU6050, and SW420
7. The software stack includes Raspberry Pi OS, Python and its libraries, FastAPIs, React Native, Postgresql, Golang, Docker, and Supabase.

SYSTEM DESIGN

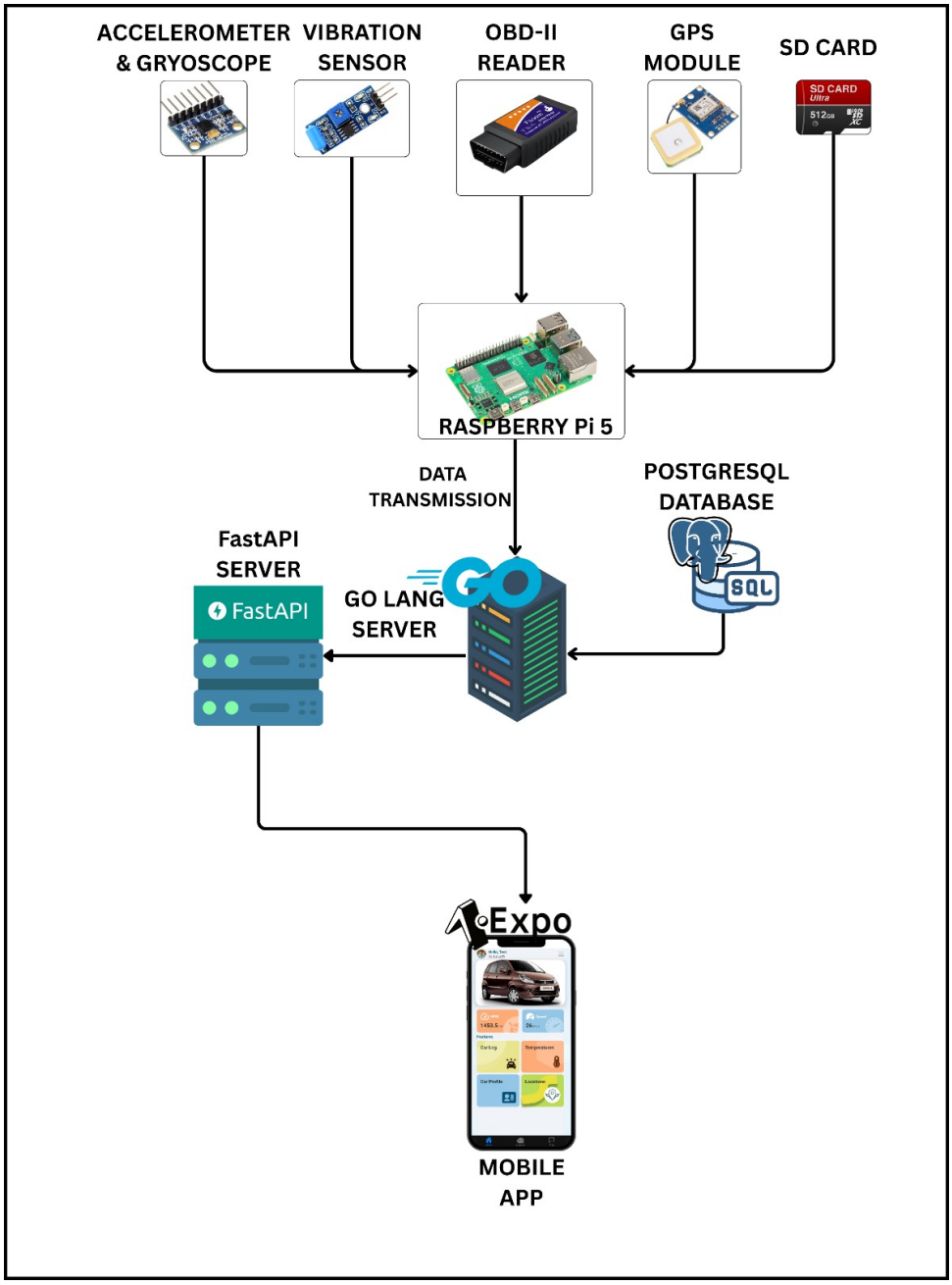

Fig. 4.1: System Design Overview

4.1 SYSTEM ARCHITECTURE OVERVIEW

The proposed vehicle diagnostic system integrates various components of hardware and software to collect real-time data for analysis. The architecture is designed to collect vehicle infor-

mation via the OBD-II port, transmit it wirelessly to a cloud server for processing, and provide a visual representation of processed data through a mobile application. The system consists of 4 primary components: Hardware Implementation, Data processing using machine learning models, Servers and Databases, and a cross-platform mobile application. A block diagram representing the overall system architecture and data flow from vehicle sensors to end user presentation is shown in Fig. 4.1. The diagram illustrates the flow of data and interactions within an automotive diagnostic and monitoring system. The system integrates multiple sensors, data processing units, databases, and APIs to achieve real-time diagnostics and data sharing. Below is a detailed explanation of the components and their interactions:

1. **Sensor and Hardware Layer:** At the core of the system lies the Raspberry Pi 5, which

Table 4.1: Modules and their Functional Descriptions

Module Name	**Description**
Accelerometer & Gyroscope (MPU-6050)	Captures motion dynamics, acceleration, and orientation data. Essential for detecting aggressive driving, collisions, rollovers, and handling analysis.
Vibration Sensor (SW-420)	Monitors abnormal vibrations that may indicate impacts, rough terrain, mechanical faults, or engine irregularities.
OBD-II Reader	Communicates with the vehicle's ECU via automotive protocols (CAN, ISO 9141, KWP2000). Extracts critical diagnostic data like RPM, throttle position, engine temperature, and fault codes.
GPS Module (NEO-6M)	Provides precise geolocation data, including latitude, longitude, speed, and timestamp, enabling route tracking, incident mapping, and trip analytics.
SD Card	Offers local and persistent storage for data logging, ensuring continuous record-keeping during network interruptions or when the vehicle is offline.

   functions as the primary edge computing device. It interfaces with multiple onboard sensors and modules

2. **Edge Device – Raspberry Pi 5** The Raspberry Pi 5 serves as the central processing and data aggregation unit at the vehicle level, collecting and processing real-time data from various onboard sensors and the vehicle's ECU. It interfaces with multiple peripherals connected through its GPIO pins and I2C bus, including a NEO-6M GPS module for acquiring precise geolocation data, an MPU-6050 accelerometer and gyroscope for capturing acceleration, orientation, and motion metrics, and an SW-420 vibration sensor to detect mechanical vibrations indicative of movement or engine activity. Additionally, the

system reads critical vehicle operational parameters such as RPM, speed, coolant temperature, and fuel levels via an OBD-II interface using a Python OBD library. The hardware connections for each sensor and module are detailed in the table below: The Raspberry

Table 4.2: Device to GPIO Pin Connections on Raspberry Pi

Device	Connection	GPIO Pins
NEO-6M GPS	VCC	Pin 17 (3.3V)
	GND	Pin 20
	TX	GPIO15 (Pin 10)
	RX	Not Connected
MPU-6050	VCC	Pin 1 (3.3V)
	GND	Pin 6
	SCL	I2C SCL (Pin 5)
	SDA	I2C SDA (Pin 3)
SW-420 Sensor	VCC	Pin 4 (5V)
	GND	Pin 14
	D0	GPIO17 (Pin 11)

Pi 5 runs two parallel threads for efficient data handling: one continuously polls OBD-II data every second, while the other simultaneously captures readings from the connected sensors. All collected data is synchronized, merged into a unified Pandas DataFrame with timestamped entries, and stored locally as a CSV file for redundancy. The system then attempts to transmit this data to a Go-based cloud server via HTTP GET requests in real-time. If network connectivity is unavailable, it temporarily saves the data to a local CSV file, ensuring no data loss by queuing the entries for batch transmission once the connection is re-established. This edge computing framework enables continuous, intelligent vehicle monitoring and secure data management, supporting essential applications such as predictive maintenance, fuel emission analysis, engine health diagnostics, and real-time fleet tracking.

3. **API Interface – Golang server** The Golang Server acts as the intermediate communication layer between the Raspberry Pi 5 edge device and the backend infrastructure, enabling structured, secure, and reliable data exchange. It exposes three RESTful endpoints: /upload, /data, and /health, each designed to handle specific interactions. The /upload endpoint, accessible via a POST request, receives CSV files containing logged telemetry data from the Raspberry Pi. Upon receiving a file, it processes the content using Go's CSV reader package, reads the records line by line, and dynamically constructs SQL queries to insert the data into a PostgreSQL database, ensuring flexibility in handling varying data formats. The /data endpoint, accessible via a GET request, executes predefined SQL queries on the database to fetch the latest stored records and returns the results as a structured HTTP response to the requester. Additionally, a lightweight /health endpoint, using a GET method, provides a quick health check mechanism for monitoring

the server's operational status. This Golang-based API server forms a robust and efficient bridge between the vehicle's edge-level data collection systems and centralized backend services, ensuring reliable data ingestion, storage, and retrieval in real-time or batch scenarios.

4. **Database Layer – PostgreSQL** The PostgreSQL Database serves as the central data repository, securely storing all structured and processed telemetry and diagnostic data collected from the vehicle. It maintains comprehensive records, including vehicle diagnostics data such as engine parameters, error codes, and sensor readings retrieved via OBD-II, along with GPS logs for tracking geospatial coordinates, route histories, and vehicle movement timestamps. Additionally, it records continuous temperature data for monitoring engine coolant and other thermal parameters, as well as sensor data like gyroscope readings and vibration events. Deployed as a Docker container within the same Docker bridge network as the Golang API server and FastAPI machine learning server, PostgreSQL ensures seamless, containerized data exchange. A dedicated database named Adam is created to organize this information, featuring structured tables for OBD data, GPS logs, gyroscope readings, vibration events, and user details, each associated with a unique user ID for secure, user-specific data management. PostgreSQL's ACID-compliant transaction handling and advanced querying capabilities guarantee data integrity, consistency, and security, making it an ideal, reliable choice for this mission-critical automotive data management solution.

5. **Backend Logic – FastAPI Server** The FastAPI Server acts as the backbone of the application, hosting the core business logic and handling machine learning-driven analytics on the collected vehicle data. It processes and analyzes critical parameters such as fuel emissions, engine health, and predictive maintenance requirements, ensuring timely and actionable insights. Additionally, it powers an integrated chatbot service for user interaction and support.
   By performing these computations at the API level, the system delivers near real-time analytics and intelligent decision-making at the edge, reducing the dependency on cloud resources and enhancing responsiveness for end users.

6. **User Interface – Mobile Application (Expo Framework)** Developed using Expo (React Native), the mobile application provides a user-friendly interface for personal vehicle owners. It enables users to:

   - View historical vehicle data.
   - Tracks trip history, average speed, top speed, fuel logs.
   - Displays real-time or historical engine temperature, coolant temperature, or outside ambient temperature.

- Tracks real-time GPS location, and nearby services like petrol pumps and repair centers with navigation.
- Offers tools for monitoring engine health, checking emission compliance, and predicting maintenance needs with detailed insights.
- Displays Car's technical and ownership details
- Provides Chat support or AI help related to car issues.

The application communicates securely with the backend servers, ensuring synchronization of data and a consistent user experience across platforms.

4.2 CHAPTER SUMMARY

1. **Sensor Integration:** The system begins with the integration of multiple sensors that capture real-time vehicle data, including parameters such as engine RPM, GPS coordinates, temperature readings, etc. These sensors transmit raw data directly to the connected Raspberry Pi5.

2. **Raspberry Pi 5 – Edge Processing Unit:** The **Raspberry Pi 5** acts as the central processor, interfacing with GPS, accelerometer, gyroscope, and vibration sensors to gather motion and location data. It also collects engine metrics like RPM, speed, and temperature via the OBD-II interface using a Python library.

3. **GoLang Server – Backend and Data Management:** The GoLang server is responsible for handling data flow, and ensuring the efficient storage of processed data into the PostgreSQL database. It also supports secure data sharing with authorized stakeholders and enables smooth communication with external systems.

4. **PostgreSQL Database – Central Data Repository:** All vehicle diagnostics, GPS logs, behavioral insights, and incident data are stored in the PostgreSQL database. Its ACID compliance ensures data integrity and reliability, while advanced querying capabilities support efficient data retrieval and analytics.

5. **FastAPI Server – API and Analytics Layer:** The **FastAPI server** functions as the application's API layer, receiving structured data from the Raspberry Pi, validating it, and performing analytics like predictive maintenance using machine learning models. The processed data is then sent to the mobile application for display.

6. **Mobile Application – User Interface Layer:** The mobile application acts as the primary user interface, connecting to the **FastAPI backend** to display real-time vehicle data, historical trends, and alerts for issues like overheating or abnormal driving behavior. It supports **role-based access**, offering tailored views for vehicle owners, fleet managers, and insurance personnel.

IMPLEMENTATION

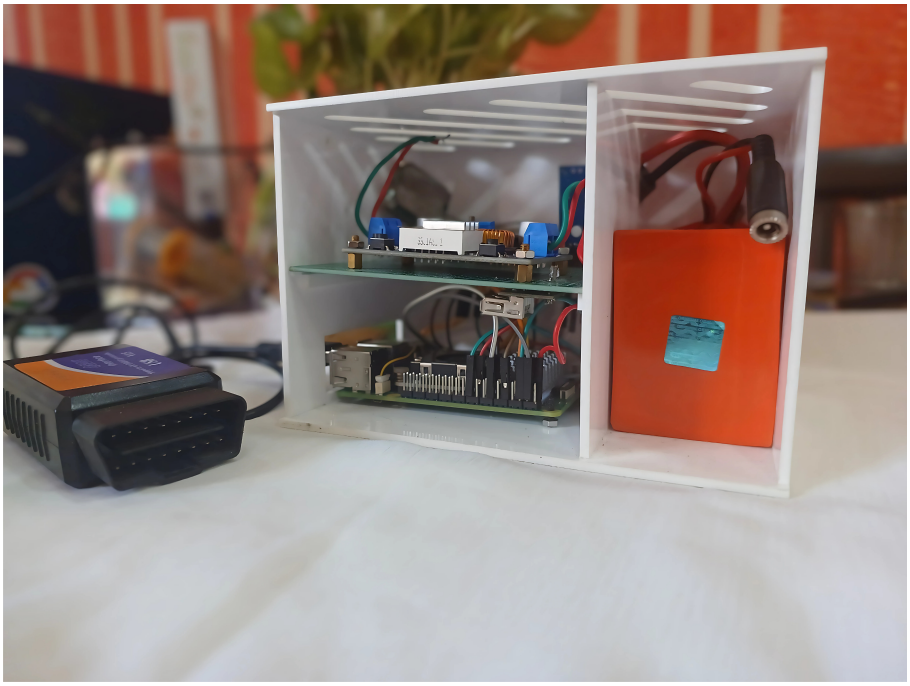

Fig. 5.1: Automotive Diagnostic And Activity Monitoring (ADAM) BOX

5.1 INTRODUCTION

This chapter discusses the implementation of our system, which consists of three primary components: the hardware setup, cloud-based storage and analysis, and a mobile application for user interaction.

5.1.1 Hardware Implementation

The hardware implementation consists of multiple sensors connected to a Raspberry Pi 5, serving as the central edge computing unit for real-time data acquisition, local processing, and cloud transmission. The system integrates several key components:

- **Raspberry Pi 5:** Acts as the central processing unit, efficiently handling real-time data acquisition, local computation, and sensor coordination.
- **GPS Module (NEO-6M):** Enables accurate vehicle location tracking and route history monitoring.

- **Vibration Sensor (SW-420):** Detects collision events and unusual vibrations indicative of accidents or aggressive driving patterns.
- **Gyroscope Module:** Monitors vehicle orientation, tilt, and angular motion for advanced motion analysis, including rollover detection and abrupt turn identification.
- **Power Management:** Connected to the vehicle battery via a relay system to prevent direct current cut-off when the car is turned off, ensuring continuous monitoring capability.

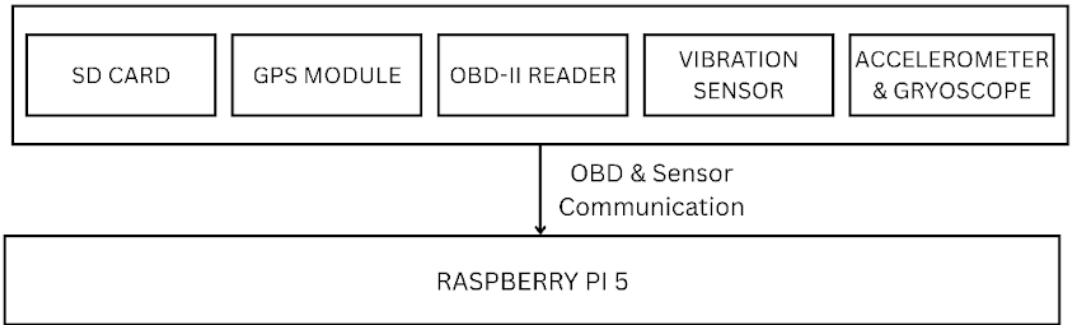

Fig. 5.2: Flow Diagram of Raspberry Pi-Based Hardware Setup

Full OBD2 Functionality

The system supports full OBD2 capabilities through its connection with the vehicle's onboard diagnostics port. These functions include reading and clearing diagnostic trouble codes (DTCs), accessing live data, viewing freeze frame data, checking readiness monitors, and performing system-level tests. Additional features include vehicle information retrieval, DTC lookup, and battery testing.

Common OBD2 Functionalities:

1. **Reading and Clearing Diagnostic Trouble Codes (DTCs):**
   - Read error codes generated by the ECU when a fault is detected.
   - Clear codes to reset indicators like the Check Engine Light.
2. **Live Data Streaming:**
   - Access real-time sensor data such as engine RPM, temperature, and vehicle speed.
   - Useful for diagnosing intermittent faults.
3. **Freeze Frame Data:**

- Snapshot of ECU data at the time of DTC generation.
- Provides context to analyze fault origin.

4. **Readiness Monitors (I/M Readiness):**
   - Shows the status of emission-related monitors.
   - Useful for inspection preparation.
5. **System Tests:**
   - O2 sensor, EVAP system, and on-board monitoring tests.
6. **Additional Functions:**
   - Retrieve VIN and calibration data.
   - Battery health and charging system checks.
   - Service light reset and built-in DTC code lookup.

CAN Message Structure

The Controller Area Network (CAN) protocol is used to retrieve and interpret data from the vehicle's electronic control units (ECUs). A typical CAN message frame consists of key fields such as identifier, data length code (DLC), and data payload, which collectively define how information is transmitted between modules in real time.

- **Identifier:** Uniquely identifies the type of message or the sender ECU.
- **DLC (Data Length Code):** Specifies the number of bytes in the data field (0–8).
- **Data Field:** Contains the actual payload, such as sensor values or control commands.

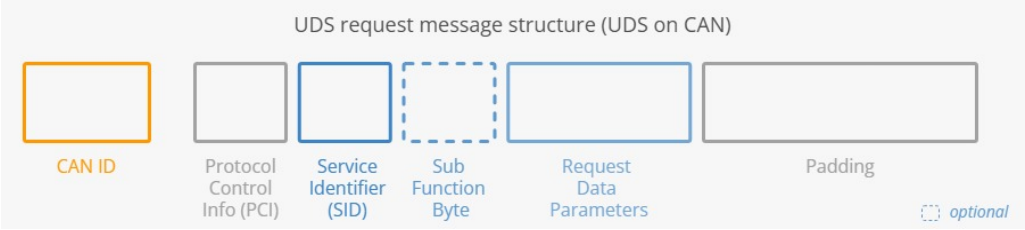

Fig. 5.3: CAN Bus Message Frame Structure

Figure 5.3 illustrates the standard format of a CAN message used by the OBD2 system and Raspberry Pi for reliable sensor communication and diagnostics.

Listing 5.1: Main Hardware Implementation Function

```
def main():
    """
    Main hardware implementation function for edge-based automotive data collection
    Demonstrates parallel sensor data acquisition, local preprocessing, and cloud transmission
    """
    global Async_conn, esp_conn, running

    print("Initializing ADAM - Automotive Data Acquisition Module")

    # Read configuration parameters
    obd_port, esp8266_port, esp8266_baud, sensor_file, temp_csv_path, data_csv_path, server_url = read_config()

    try:
        # Initialize hardware connections
        Async_conn = Async_connection(obd_port)  # OBD-II interface
        esp_conn = Esp8266_conn(esp8266_port, esp8266_baud)  # ESP8266 sensor module

        print(f"Hardware initialized - OBD: {obd_port} | ESP8266: {esp8266_port}")

        # Main data collection loop
        iteration = 1
        while running:
            print(f"\n--- Data Collection Cycle {iteration} ---")

            # Parallel sensor data acquisition
            results = {}

            # Create concurrent threads for multi-sensor data collection
            obd_thread = threading.Thread(
                target=collect_obd_data,
```

```
                args=(Async_conn, 1, sensor_file, results)
            )
            esp_thread = threading.Thread(
                target=collect_serial_data,
                args=(esp_conn, results)
            )

            # Execute parallel data collection
            obd_thread.start()
            esp_thread.start()
            obd_thread.join()
            esp_thread.join()

            # Local preprocessing and data fusion
            if 'obd_df' in results and 'serial_df' in results:
                combined_df = merge_dataframes(results['obd_df'], results['serial_df'])

                # Edge processing: Save locally with timestamp
                combined_df.to_csv(temp_csv_path, index=False)
                append_to_data_csv(temp_csv_path, data_csv_path)

                # Cloud transmission for centralized analysis
                upload_result = upload_to_server(temp_csv_path, server_url)

                print(f"Cycle {iteration} complete - Data processed and transmitted")

            iteration += 1
            time.sleep(1)  # Regular interval collection

    except Exception as e:
        print(f"Hardware implementation error: {e}")
    finally:
        # Cleanup hardware connections
        if Async_conn: Async_conn.close()
```

```
65          if esp_conn: esp_conn.close()
66
67  def merge_dataframes(obd_df, serial_df):
68      """
69      Local preprocessing: Merge multi-sensor data with
           timestamp synchronization
70      """
71      obd_df = obd_df.reset_index(drop=True)
72      serial_df = serial_df.reset_index(drop=True)
73
74      # Add synchronized timestamp for data correlation
75      timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
76      obd_df["MERGED_TIMESTAMP"] = timestamp
77
78      # Horizontal concatenation for unified sensor data
79      combined_df = pd.concat([obd_df, serial_df], axis=1)
80      return combined_df
```

The Raspberry Pi collected sensor data at regular intervals, pre-processed it locally, and transmitted the information to a cloud backend for storage and analysis. This edge setup ensured low-latency data capture and immediate response to critical driving events.

5.2 GOLANG

Once data is collected and processed locally by the Raspberry Pi 5 edge device, it is transmitted to a Golang-based server which acts as a middleware communication layer. This Go server ensures the reliable and secure transmission of structured telemetry data from edge to backend systems, enabling near real-time monitoring and decision-making. The Go server exposes a set of RESTful API endpoints for interacting with various components of the data pipeline. These include:

1. **/upload:** Handles incoming CSV files via POST requests. Data is parsed using Go's CSV reader libraries, and SQL queries are dynamically constructed for storage in the PostgreSQL database.

Listing 5.2: Upload CSV Route Handler

```
1  func UploadHandler(c *fiber.Ctx) error {
2      file, _ := c.FormFile("file")
3      f, _ := file.Open()
4      defer f.Close()
5
```

```
    r := csv.NewReader(f)
    header, _ := r.Read()
    records, _ := r.ReadAll()

    table := "telemetry"
    cols := strings.Join(header, " TEXT, ") + " TEXT"
    query := fmt.Sprintf(
        "CREATE TABLE IF NOT EXISTS %s (%s);",
        table, cols)

    conn, _ := database.GetPool().
        Acquire(context.Background())
    defer conn.Release()
    conn.Exec(context.Background(), query)

    placeholders := make([]string, len(header))
    for i := range header {
        placeholders[i] = fmt.Sprintf("$%d", i+1)
    }

    insert := fmt.Sprintf(
        "INSERT INTO %s (%s) VALUES (%s)",
        table,
        strings.Join(header, ", "),
        strings.Join(placeholders, ", "),
    )

    for _, rec := range records {
        conn.Exec(
            context.Background(),
            insert,
            toIfaceSlice(rec)...,
        )
    }
    return c.SendStatus(200)
}
```

2. **/data:** Allows GET requests to fetch the most recent or specific historical records from the database. This enables dashboards, apps, or other services to retrieve and display relevant vehicle data.

Listing 5.3: Data Retrieval Route Handler

```
func FetchHandler(c *fiber.Ctx) error {
    pool := database.GetPool()
    conn, err := pool.Acquire(context.Background())
    if err != nil {
        log.Printf("DB connection error: %v", err)
        return c.Status(500).JSON(
            fiber.Map{"error": "DB error"})
    }
    defer conn.Release()

    table := "dataset"
    query := `SELECT * FROM ` + table
    rows, err := conn.Query(context.Background(), query)
    if err != nil {
        log.Printf("Query error: %v", err)
        return c.Status(500).JSON(
            fiber.Map{"error": "Query error"})
    }
    defer rows.Close()

    fields := rows.FieldDescriptions()
    columns := make([]string, len(fields))
    for i, f := range fields {
        columns[i] = string(f.Name)
    }

    var result []map[string]interface{}
    for rows.Next() {
        row := make([]interface{}, len(columns))
        ptrs := make([]interface{}, len(columns))
        for i := range row {
            ptrs[i] = &row[i]
        }
        if err := rows.Scan(ptrs...); err != nil {
            log.Printf("Scan error: %v", err)
            return c.Status(500).JSON(
                fiber.Map{"error": "Scan error"})
        }
        rowMap := make(map[string]interface{})
        for i, col := range columns {
```

```
            if v, ok := row[i].([]uint8); ok {
                rowMap[col] = string(v)
            } else {
                rowMap[col] = row[i]
            }
        }
        result = append(result, rowMap)
    }

    if err := rows.Err(); err != nil {
        log.Printf("Row iteration error: %v", err)
        return c.Status(500).JSON(
            fiber.Map{"error": "Read error"})
    }

    return c.JSON(fiber.Map{
        "data":    result,
        "count":   len(result),
        "columns": columns,
    })
}
```

3. **/health:** A lightweight endpoint that returns the server's health status, used for monitoring and diagnostics.

Listing 5.4: Health Check Route Handler

```
var startTime = time.Now()

func HealthHandler(c *fiber.Ctx) error {
    healthData := map[string]interface{}{
        "status":    "healthy",
        "uptime":    utils.GetUptime(startTime),
        "timestamp": time.Now().
            Format(time.RFC3339),
    }

    return c.Status(fiber.StatusOK).
        JSON(healthData)
}
```

Through these endpoints, the Go server functions as the key intermediary for reliable, flexible, and secure data ingestion into the centralized storage system.

5.3 POSTGRES DB

The collected data is stored in a PostgreSQL (PGSQL) database for future processing and analysis. This database serves as the central repository for storing vehicle data, including various parameters like engine temperature, fuel consumption, speed, and more. The structured format of the database allows for efficient querying and data retrieval. All incoming telemetry and diagnostic data are stored in a PostgreSQL (PGSQL) relational database, which acts as the core data repository for the ADAM system. PostgreSQL is selected due to its enterprise-level capabilities, including full support for ACID transactions, advanced indexing techniques, query optimization, and secure storage features. The PostgreSQL database is designed with a normalized schema that efficiently captures and organizes various datasets such as: Engine diagnostic parameters and fault codes

- GPS and route tracking logs
- Vehicle movement and sensor activity
- Temperature metrics and thermal events
- User metadata and access logs

Structured storage facilitates rapid query response times and easy retrieval of records for downstream analytics and machine learning tasks. This centralized repository ensures long-term archival, auditability, and scalability as data volume grows.

5.4 FASTAPI

Once stored, the diagnostic data is accessed by a FastAPI-based application layer. This server plays a central role in processing data, serving APIs to external interfaces, and executing real-time and scheduled analysis tasks. One of its most critical roles is the application of Machine Learning (ML) models trained on historical vehicle data to derive predictive insights.

1. **Emissions Diagnostic:** The emissions router (`routers/emissions.py`) processes real-time sensor data to determine emission compliance status using the EnhancedCarEmissionsMLModel. When a request is received, the router accepts CSV uploads or JSON payloads containing sensor readings such as CO levels, NOx concentrations, and particulate matter measurements. The data is validated using Pydantic models before being passed to the emissions model, which preprocesses the sensor values, applies Indian emission standard thresholds, and generates compliance classifications using a trained

Random Forest classifier. The router returns structured JSON responses containing compliance status, confidence scores, and detailed sensor analysis, along with optional confusion matrix visualizations for model performance assessment.

2. **Engine Health Diagnostic:** As part of the **ADAM system**, an engine health monitoring module was built using a **Random Forest Classifier** trained on OBD-II vehicle telemetry. The model classifies engine condition as *"Good"* or *"Needs Attention"*, enabling predictive maintenance and improving vehicle safety.

Workflow

- OBD-II data (RPM, throttle position, temperature, voltage, misfire counts, etc.) is preprocessed and labeled as healthy or faulty.
- Data split: **70% training**, **30% testing**.
- The Random Forest model is trained and tested, achieving **92% accuracy**, with perfect precision and recall.
- Feature importance analysis identified RPM, coolant temperature, throttle position, misfire counts, and control module voltage as key predictors.

Table 5.1: Internal Evaluation on Expert Data (Engine Health Classification)

Class	**Precision**	**Recall**	**F1-Score**	**Support**
bb (Needs Attention)	0.85	0.85	0.85	48
gg (Good)	0.95	0.95	0.95	137
Accuracy	0.92 (185 samples)			
Macro Average	0.90	0.90	0.90	185
Weighted Average	0.92	0.92	0.92	185

Router Function

The `engine_health.py` router processes incoming OBD data, runs it through the trained model, and returns a health status report with:

- Predicted engine condition
- Anomaly detection insights
- Maintenance recommendations

A distribution graph visualizes occurrences of predicted engine conditions across the dataset, as shown in **Figure 5.4**.

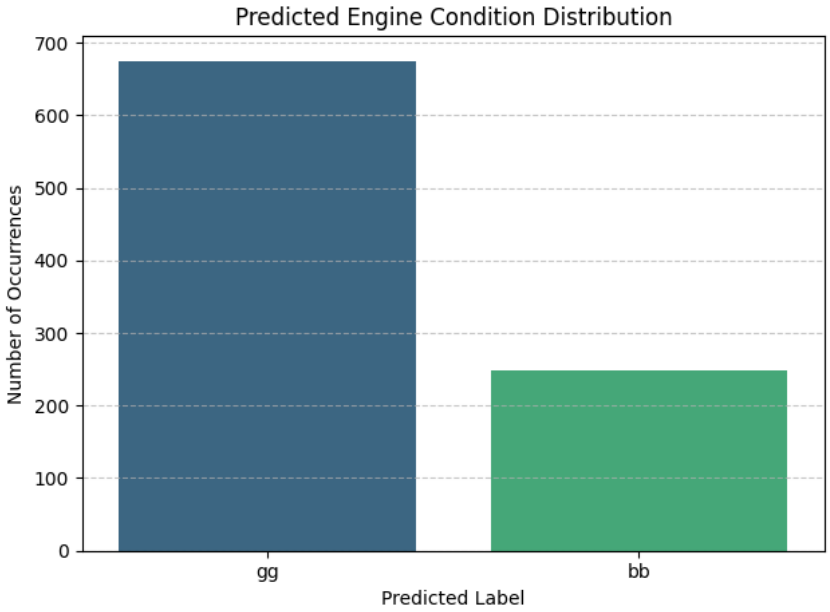

Fig. 5.4: Predicted Engine Condition Distribution (Good vs. Needs Attention)

3. **Predictive Maintenance System:** The **Predictive Maintenance Router** (`routers/predictive\_maintenance.py`) serves as a cornerstone module within the **ADAM system**, engineered to deliver comprehensive fleet maintenance analytics through automated diagnostic workflows. The system integrates real-time vehicle telemetry processing with rule-based maintenance scheduling to ensure optimal fleet operational reliability.

5.4.1 System Architecture and Data Processing

The predictive maintenance module employs a multi-stage processing pipeline that transforms raw sensor data into actionable maintenance insights. The core implementation leverages the `VehicleMaintenance` class to systematically evaluate vehicle health metrics including mileage progression, engine operational parameters, sensor performance indicators, and historical usage patterns.

Listing 5.5: Main Hardware Implementation Function

```
1 @router.get("/diagnose")
2 def generate_predictive_diagnosis():
3     """
4     Comprehensive endpoint that runs all maintenance
        diagnostics and generates a detailed report.
5     """
6     global maintenance_reports, predictive_report
7
```

```
    try:
        # Step 1: Fetch Data
        print("\n[1/4] Fetching maintenance data...")
        conn = connect_to_db()
        query = "SELECT * FROM kaiserskoda;"
        df = pd.read_sql(query, conn)
        conn.close()
        print(" Data fetched successfully")

        # Step 2: Preprocess
        print("\n[2/4] Preprocessing maintenance data..."
            )
        processed_df = preprocess_data(df)
        print(" Preprocessing complete")

        # Step 3: Generate Vehicle Reports
        print("\n[3/4] Analyzing vehicle conditions...")
        reports = []
        maintenance_statuses = {
            'Critical': 0,
            'Warning': 0,
            'Good': 0
        }

        for idx, row in processed_df.iterrows():
            try:
                vehicle = VehicleMaintenance(
                    distance_w_mil=row["distance_w_mil"],
                    time_in_years=row["time_in_years"],
                    speed=row["speed"],
                    engine_load=row["engine_load"],
                    coolant_temp=row["coolant_temp"],
                    run_time=row["run_time"],
                    throttle_pos=row["throttle_pos"],
                    control_module_voltage=row["
                        control_module_voltage"],
                    fuel_level=row["fuel_level"],
                    short_fuel_trim_1=row["
                        short_fuel_trim_1"],
                    long_fuel_trim_1=row["
                        long_fuel_trim_1"],
```

```
45                      ambiant_air_temp=row["
                            ambiant_air_temp"],
46                      o2_sensors=row["o2_sensors"],
47                      catalyst_temp_b1s1=row["
                            catalyst_temp_b1s1"]
48                  )
49
50                  report = vehicle.generate_report()
51
52                  # Determine vehicle status based on
                        conditions
53                  critical_conditions = [
54                      "immediately" in str(status).lower()
55                      for status in report.values()
56                      if isinstance(status, str)
57                  ]
58                  warning_conditions = [
59                      "soon" in str(status).lower()
60                      for status in report.values()
61                      if isinstance(status, str)
62                  ]
63
64                  if any(critical_conditions):
65                      report['overall_status'] = 'Critical'
66                      maintenance_statuses['Critical'] += 1
67                  elif any(warning_conditions):
68                      report['overall_status'] = 'Warning'
69                      maintenance_statuses['Warning'] += 1
70                  else:
71                      report['overall_status'] = 'Good'
72                      maintenance_statuses['Good'] += 1
73
74                  reports.append(report)
75
76              except Exception as e:
77                  logging.error(f"Error processing vehicle
                        {idx}: {str(e)}")
78                  continue
79
80          if not reports:
81              raise ValueError("No valid reports were
```

```
                generated")
82
83          maintenance_reports = pd.DataFrame(reports)
84
85          # Step 4: Generate Summary Report
86          print("\n[4/4] Generating comprehensive
               maintenance report...")
87
88          summary_stats = {
89              "timestamp": "2025-01-19 09:21:26",
90              "total_vehicles": len(reports),
91              "status_distribution": maintenance_statuses,
92              "critical_systems": {
93                  "engine": maintenance_reports['Engine
                       Load'].mean() if 'Engine Load' in
                       maintenance_reports else 0,
94                  "coolant": maintenance_reports[
95                      'Coolant Temperature'].mean() if '
                           Coolant Temperature' in
                           maintenance_reports else 0,
96                  "fuel": maintenance_reports['Fuel Level'
                       ].mean() if 'Fuel Level' in
                       maintenance_reports else 0,
97                  "brake_wear": maintenance_reports[
98                      'Brake Pad Wear'].mean() if 'Brake
                           Pad Wear' in maintenance_reports
                           else 0
99              },
100             "maintenance_required": {
101                 "immediate_attention":
                       maintenance_statuses['Critical'],
102                 "soon_attention": maintenance_statuses['
                       Warning'],
103                 "good_condition": maintenance_statuses['
                       Good']
104             }
105         }
106
107         # Store the full report
108         predictive_report = maintenance_reports.copy()
109         predictive_report['analysis_timestamp'] = "
```

```
            2025-01-19 09:21:26"
110
111         print("\nMaintenance Analysis Summary:")
112         print(f"Total Vehicles Analyzed: {summary_stats['
                total_vehicles']}")
113         print(f"Critical Issues: {maintenance_statuses['
                Critical']}")
114         print(f"Warnings: {maintenance_statuses['Warning
                ']}")
115         print(f"Good Condition: {maintenance_statuses['
                Good']}")
116
117         return {
118             "summary": summary_stats,
119             "report_preview": predictive_report.head(10).
                    to_dict(orient="records"),
120             "timestamp": "2025-01-19 09:21:26"
121         }
122
123     except Exception as e:
124         logging.error(f"Error in predictive diagnosis: {
                str(e)}")
125         raise HTTPException(
126             status_code=500,
127             detail=f"Error generating predictive
                    maintenance report: {str(e)}"
128         )
```

5.4.2 Rule-Based Diagnostic Framework

Distinguished from machine learning approaches utilized in other system modules, the predictive maintenance router implements deterministic, threshold-based algorithms derived from OEM specifications and empirical operational data. This methodology ensures consistent, interpretable diagnostic outcomes across diverse vehicle configurations and operational environments.

The diagnostic evaluation framework, as depicted in **Figure 5.5**, processes real-time sensor data through the OBD-II interface and applies conditional assessment protocols across critical vehicle subsystems:

- **Engine Lubrication System:** Engine run-time exceeding 7,500 operational units

triggers automatic oil change scheduling with immediate priority classification.

- **Air Intake Management:** Intake manifold pressure readings outside the optimal 95–105 kPa range indicate air filtration system degradation requiring filter replacement.
- **Ignition System Integrity:** Non-zero misfire count detection signals spark plug deterioration, prompting ignition component inspection and replacement scheduling.
- **Electrical System Health:** Control module voltage measurements below the critical 11.5V threshold indicate battery system compromise requiring immediate attention.
- **Throttle Response Calibration:** Throttle position sensor variance exceeding 10% deviation indicates calibration drift necessitating system recalibration procedures.
- **Fuel Mixture Optimization:** Oxygen sensor voltage readings outside the standard 0.8–1.2V operational window suggest air-fuel mixture irregularities requiring sensor maintenance.
- **Thermal Management:** Coolant temperature measurements exceeding 110°C operational threshold indicate cooling system inefficiency requiring immediate intervention.
- **Emissions Control Monitoring:** Any active evaporative system diagnostic trouble codes (DTCs) automatically flag emission control system maintenance requirements.

5.4.3 Maintenance Classification and Reporting

The system employs a tri-tier classification scheme to prioritize maintenance interventions based on operational criticality and safety implications. Each vehicle assessment results in one of three status classifications:

Critical Status: Vehicles exhibiting conditions requiring immediate maintenance intervention to prevent operational failure or safety compromise.

Warning Status: Vehicles demonstrating early-stage component degradation requiring scheduled maintenance within the next service interval.

Good Status: Vehicles operating within normal parameters with no immediate maintenance requirements identified.

Upon completion of the diagnostic evaluation cycle, the system generates comprehensive maintenance documentation including component-specific recommendations, estimated replacement timelines, cost projections, and priority-based scheduling suggestions. This

systematic approach ensures fleet managers receive actionable intelligence for proactive maintenance planning while minimizing unscheduled downtime and operational disruptions.

The rule-based diagnostic strategy provides superior transparency and reliability compared to black-box machine learning alternatives, enabling maintenance technicians to understand the precise reasoning behind each recommendation while maintaining consistent diagnostic accuracy across the entire vehicle fleet.

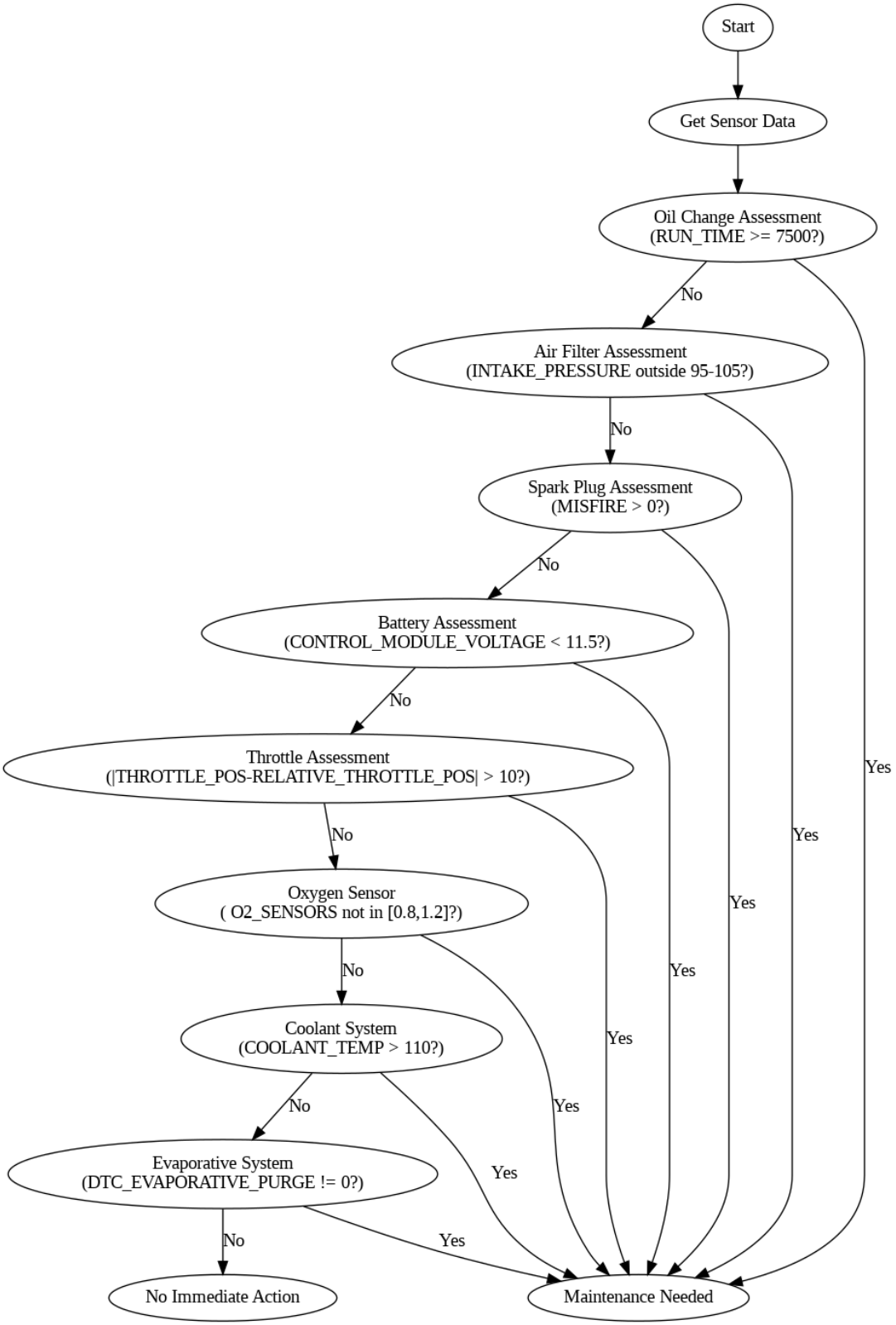

Fig. 5.5: Comprehensive Flowchart of the Predictive Maintenance Router Diagnostic Process

5.5 MOBILE APPLICATION

The **ADAM system's mobile application** home screen is implemented using **React Native** and designed to offer users quick access to critical vehicle telemetry, car logs, location tracking, and performance analytics in an intuitive, visually engaging format.

5.5.1 User Interface Structure

The home screen comprises several visually segmented sections:

- **Profile Header:** Displays the user's profile picture, name, and the current date dynamically.
- **Vehicle Image Display:** A featured image of the user's registered vehicle at the top of the interface.
- **Live Metrics:** Real-time metrics for RPM and speed displayed within color-coded metric cards.
- **Feature Navigation Grid:** A grid of interactive feature cards providing access to car logs, temperature monitoring, car profile management, and location tracking.

5.5.2 Component Details

Profile Header

A header displaying:

- User's profile picture.
- Greeting message with the user's name fetched from the authentication context.
- Current date formatted in `DD-MMM-YYYY`.
- A navigation icon linking to the user's profile.

Vehicle Image

An image component displaying the vehicle's registered image at a fixed height of `200px`.

Metric Cards

Two cards display real-time data:

1. **RPM:** Value in `rpm` with a background image icon faded into the card.
2. **Speed:** Value in `km/h` with a speedometer graphic in the background.

Feature Grid

A grid of four interactive cards:

- **Car Log:** Opens a vehicle log report.
- **Temperatures:** Displays current engine and ambient temperatures.
- **Car Profile:** Displays the registered vehicle's detailed profile.
- **Locations:** Accesses live or historical vehicle locations via integrated mapping services.

Each card includes:

- Title label.
- Thematic icon.
- Lightly opaque background graphics.
- Navigation triggers linked to appropriate routes.

5.5.3 Navigation and Routing

The application uses the `expo-router` library for dynamic routing. Each `TouchableOpacity` feature card redirects users to a specific route:

- `/log/clog` for Car Log.
- `/temp/temp` for Temperatures.
- `/(profile)` for Car Profile.
- `/maps/1` for Locations.

5.5.4 Illustrative Layout

An illustration of the home screen layout is shown below.

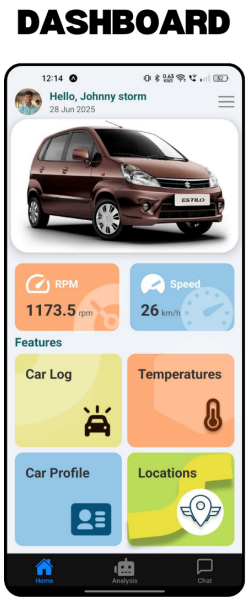

Fig. 5.6: Dashboard of the ADAM mobile application

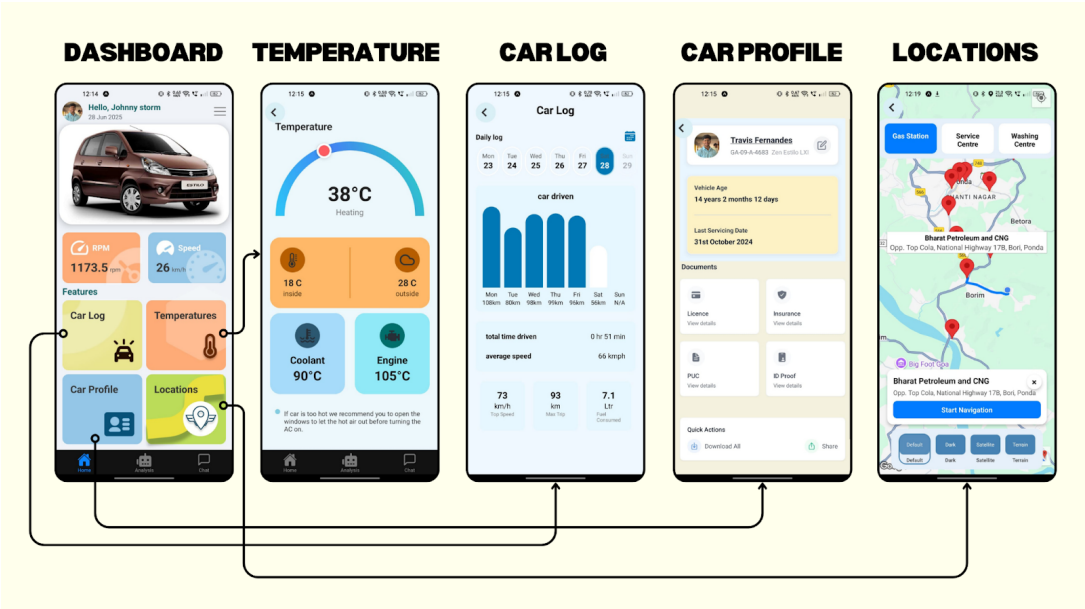

Fig. 5.7: The dashboard includes direct navigation to four essential modules

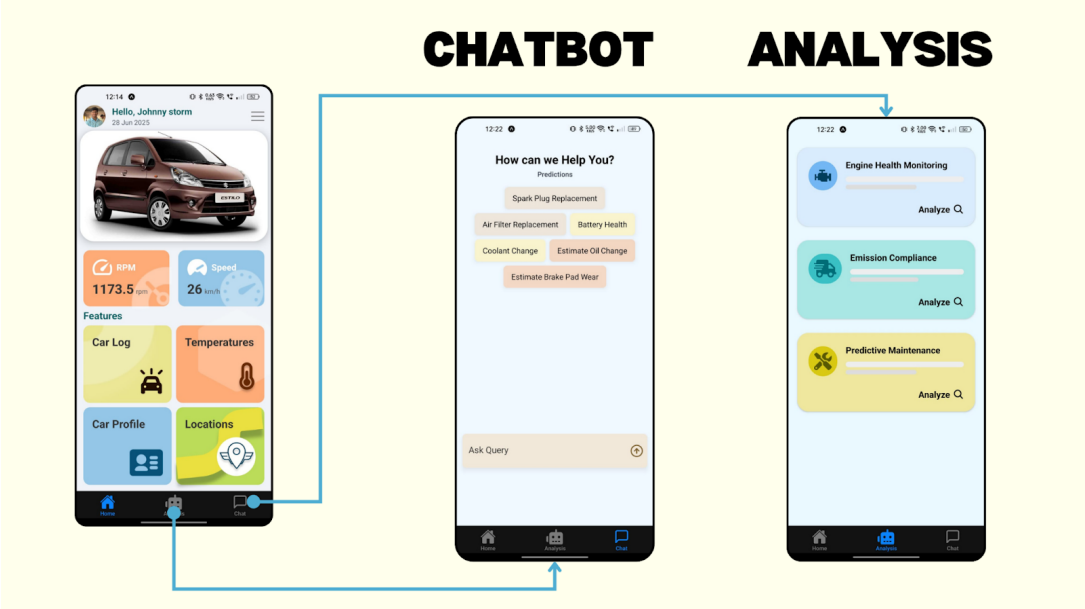

Fig. 5.8: The bottom tab of the ADAM mobile application provides quick access to key features

The following snippet shows the implementation of the `Analysis` screen in the ADAM mobile application. It uses `React Native`, `useFetch` hooks for data fetching, and `DataCard` components to display real-time telemetry and diagnostics:

Listing 5.6: React Native Analysis Screen Component

```
1  import { View, ScrollView } from 'react-native';
2  import React from 'react';
3  import { SafeAreaView } from 'react-native-safe-area-context'
       ;
4  import { MaterialCommunityIcons, FontAwesome5 } from '@expo/
       vector-icons';
5  import DataCard from '@/components/analysis/DataCard';
6  import { HealthMonitoringDisplay, EmissionsDisplay,
       PredictiveMaintenanceDisplay } from '@/components/analysis
       /DataDisplay';
7  import { useFetch } from '@/hooks/useFetch';
8  import { analysisService } from '@/services/api';
9  import { cardThemes } from '@/constants/cardThemes';
10
11 const Analysis = () => {
12   const { data: healthMonitoringData, loading:
         loadingHealthMonitoring,
```

```
13      disabled: healthMonitoringDisabled, fetchData:
            fetchHealthMonitoringData }
14      = useFetch(analysisService.fetchHealthMonitoringData);
15
16    const { data: emissionsData, loading: loadingEmissions,
17      disabled: emissionsDisabled, fetchData:
            fetchEmissionsData }
18      = useFetch(analysisService.fetchEmissionsData);
19
20    const { data: predictiveMaintenanceData, loading:
          loadingPredictiveMaintenance,
21      disabled: predictiveMaintenanceDisabled, fetchData:
            fetchPredictiveMaintenanceData }
22      = useFetch(analysisService.fetchPredictiveMaintenanceData
            );
23
24    return (
25      <SafeAreaView style={{ flex: 1, backgroundColor: '#EAF8FF
            ' }}>
26        <ScrollView style={{ padding: 20 }}>
27          <DataCard
28            title={cardThemes.healthMonitoring.title}
29            icon={<MaterialCommunityIcons name='engine' size
                ={32} color="#003f5c" />}
30            bgColor={cardThemes.healthMonitoring.bgColor}
31            iconBgColor={cardThemes.healthMonitoring.
                iconBgColor}
32            loading={loadingHealthMonitoring}
33            disabled={healthMonitoringDisabled}
34            onFetch={fetchHealthMonitoringData}>
35            <HealthMonitoringDisplay data={healthMonitoringData
                } />
36          </DataCard>
37
38          <DataCard
39            title={cardThemes.emissions.title}
40            icon={<FontAwesome5 name='shipping-fast' size={32}
                color="#006D77" />}
41            bgColor={cardThemes.emissions.bgColor}
42            iconBgColor={cardThemes.emissions.iconBgColor}
43            loading={loadingEmissions}
```

```
44            disabled={emissionsDisabled}
45            onFetch={fetchEmissionsData}>
46            <EmissionsDisplay data={emissionsData} />
47          </DataCard>
48
49          <DataCard
50            title={cardThemes.predictiveMaintenance.title}
51            icon={<FontAwesome5 name='tools' size={32} color="
                 #5A5A00" />}
52            bgColor={cardThemes.predictiveMaintenance.bgColor}
53            iconBgColor={cardThemes.predictiveMaintenance.
                 iconBgColor}
54            loading={loadingPredictiveMaintenance}
55            disabled={predictiveMaintenanceDisabled}
56            onFetch={fetchPredictiveMaintenanceData}>
57            <PredictiveMaintenanceDisplay data={
                 predictiveMaintenanceData} />
58          </DataCard>
59        </ScrollView>
60      </SafeAreaView>
61    );
62  };
63
64  export default Analysis;
```

The following snippet presents the implementation of the `Temperature` screen within the ADAM mobile application. This screen is built using `React Native` and displays dynamic real-time temperature data for various vehicle components such as the engine, coolant, and cabin environment. It features an interactive arc meter created with `react-native-svg`, along with graphical indicators for inside and outside temperatures, coolant temperature, and engine temperature. The interface also includes helpful recommendations for the user based on current thermal readings to improve driving safety and comfort.

Listing 5.7: React Native Temperature Screen Component

```
1  import { router } from 'expo-router';
2  import React from 'react';
3  import { View, Text, SafeAreaView, TouchableOpacity, Image }
      from 'react-native';
4  import Svg, { Path, Defs, LinearGradient as SvgLinearGradient
      , Stop } from 'react-native-svg';
5  import cloud from "@/assets/images/cloud.png";
```

```
6  import thermo from "@/assets/images/thermo.png";
7  import temo from "@/assets/images/temo.png";
8  import eng from "@/assets/images/eng.png";
9
10 export default function TemperatureScreen() {
11   const Circle: React.FC<{ color: string; size?: number }> =
        ({ color, size = 50 }) => (
12     <View style={{ width: size, height: size, borderRadius:
          size / 2, backgroundColor: color }} />
13   );
14
15   const BackArrow = () => (
16     <View style={{
17       width: 12, height: 12,
18       borderLeftWidth: 3, borderBottomWidth: 3,
19       borderColor: '#333',
20       transform: [{ rotate: '45deg' }]
21     }} />
22   );
23
24   const ArcMeter = ({ temperature = 38 }) => {
25     const radius = 140, strokeWidth = 18;
26     const centerX = 160, centerY = 160;
27     const width = centerX * 2, height = centerY + strokeWidth
          ;
28     const minTemp = 0, maxTemp = 100;
29     const clampedTemp = Math.max(minTemp, Math.min(maxTemp,
          temperature));
30     const angle = Math.PI * (1 - (clampedTemp - minTemp) / (
          maxTemp - minTemp));
31     const knobX = centerX + radius * Math.cos(angle);
32     const knobY = centerY - radius * Math.sin(angle);
33     const startAngle = Math.PI, endAngle = 0;
34     const startX = centerX + radius * Math.cos(startAngle);
35     const startY = centerY + radius * Math.sin(startAngle);
36     const endX = centerX + radius * Math.cos(endAngle);
37     const endY = centerY + radius * Math.sin(endAngle);
38     const largeArcFlag = endAngle - startAngle <= Math.PI ? '
          0' : '1';
39     const arcPath = `M ${startX} ${startY} A ${radius} ${
          radius} 0 ${largeArcFlag} 1 ${endX} ${endY}`;
```

```
40
41      return (
42        <View style={{ alignItems: 'center', marginVertical: 30
              }}>
43          <Svg height={height} width={width}>
44            <Defs>
45              <SvgLinearGradient id="grad" x1="0" y1="1" x2="1"
                    y2="0">
46                <Stop offset="0" stopColor="#2EC4F7" />
47                <Stop offset="0.5" stopColor="#4285F4" />
48                <Stop offset="1" stopColor="#2CD5C4" />
49              </SvgLinearGradient>
50            </Defs>
51            <Path d={arcPath} stroke="url(#grad)" strokeWidth={
                  strokeWidth} fill="none" />
52          </Svg>
53
54          <View style={{
55            position: 'absolute', top: knobY - 15, left: knobX
                  - 15,
56            width: 30, height: 30, borderRadius: 15,
                  backgroundColor: '#FF5A5F',
57            borderWidth: 4, borderColor: 'white',
58            shadowColor: '#000', shadowOffset: { width: 0,
                  height: 2 },
59            shadowOpacity: 0.2, shadowRadius: 4, elevation: 5
60          }} />
61        </View>
62      );
63    };
64
65    return (
66      <SafeAreaView style={{ flex: 1, backgroundColor: '#EDF8FD
          ', padding: 20, alignItems: 'center' }}>
67        <TouchableOpacity style={{ position: 'absolute', top:
            40, left: 10, zIndex: 10 }} onPress={() => router.
            back()}>
68          <View style={{ width: 40, height: 40, borderRadius:
              20, backgroundColor: '#D9F0F7', justifyContent: '
              center', alignItems: 'center' }}>
69            <BackArrow />
```

```
70          </View>
71        </TouchableOpacity>
72
73        <Text style={{ fontSize: 20, fontWeight: '600', color:
              '#222', alignSelf: 'flex-start', marginBottom: 30,
              position: 'relative', left: 10, top: 54 }}>
              Temperature</Text>
74        <ArcMeter temperature={38} />
75        <Text style={{ fontSize: 40, fontWeight: '700', color:
              '#222', top: -120 }}>38degC</Text>
76        <Text style={{ fontSize: 16, color: '#777',
              marginBottom: 20, top:-120 }}>Heating</Text>
77      </SafeAreaView>
78    );
79  }
```

5.5.5 Chatbot for User Interaction

An integrated chatbot allows users to interact with the system and inquire about their vehicle's performance, health, and other important metrics. This feature ensures users have a conversational interface to obtain useful insights about their vehicle without navigating through complex menus.

To make the system more user-friendly and accessible, the **ADAM system** incorporates a natural language chatbot interface. This intelligent assistant allows users to interact with the system via text-based queries, avoiding the need for manual navigation.

Users can ask questions such as:

- *"What is the current fuel efficiency of my vehicle?"*
- *"Are there any upcoming maintenance needs?"*
- *"Is my vehicle emitting above permissible limits?"*
- *"How is the engine health right now?"*

The chatbot processes these queries by retrieving relevant data from the PostgreSQL database through the FastAPI backend and provides concise, human-readable responses. It enhances the user experience by offering on-demand access to critical insights regarding their vehicle's condition and maintenance schedule.

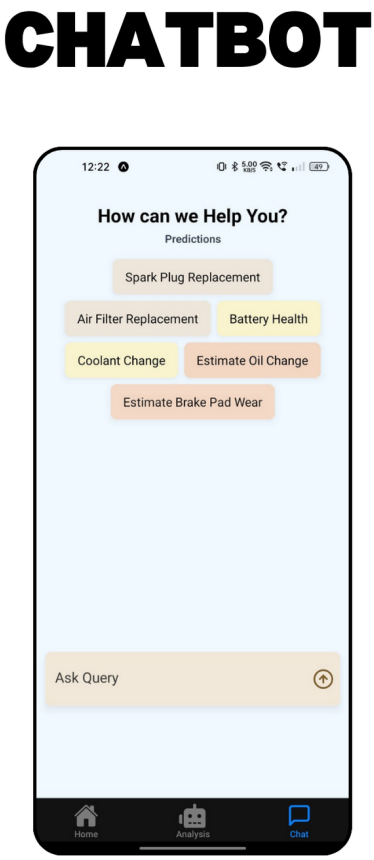

Fig. 5.9: Chatbot Interaction Interface on Mobile Device

Figure 5.9 illustrates the chatbot interface on a mobile device, showcasing how users can conveniently interact with the ADAM system using natural language queries.

5.5.6 Car Profile and Document Management System

The **Car Profile and Document Management** module in the **ADAM mobile application** serves as a centralized hub for managing and accessing all essential vehicle-related documents. This system ensures vehicle owners have streamlined visibility of their documentation status, expiry dates, and secure storage in one accessible location.

Car Profile Overview Screen

The primary screen in this module provides a summary of the vehicle and serves as the entry point for detailed document management. Key features of this screen include:

- **Owner Details:** Displays the user's name, registered vehicle number, and model (e.g., Zen Estilo LXI).
- **Vehicle Age:** Automatically calculates and displays the vehicle's current age based on registration data.

- **Last Servicing Date:** Shows the most recent servicing date, helping users track maintenance intervals.
- **Document Tiles:** Four distinct sections allow access to specific documents:
  - Licence
  - Insurance
  - PUC (Pollution Under Control)
  - ID Proof

  Each tile includes a *View Details* button that leads to an expanded screen for uploading or reviewing that particular document.

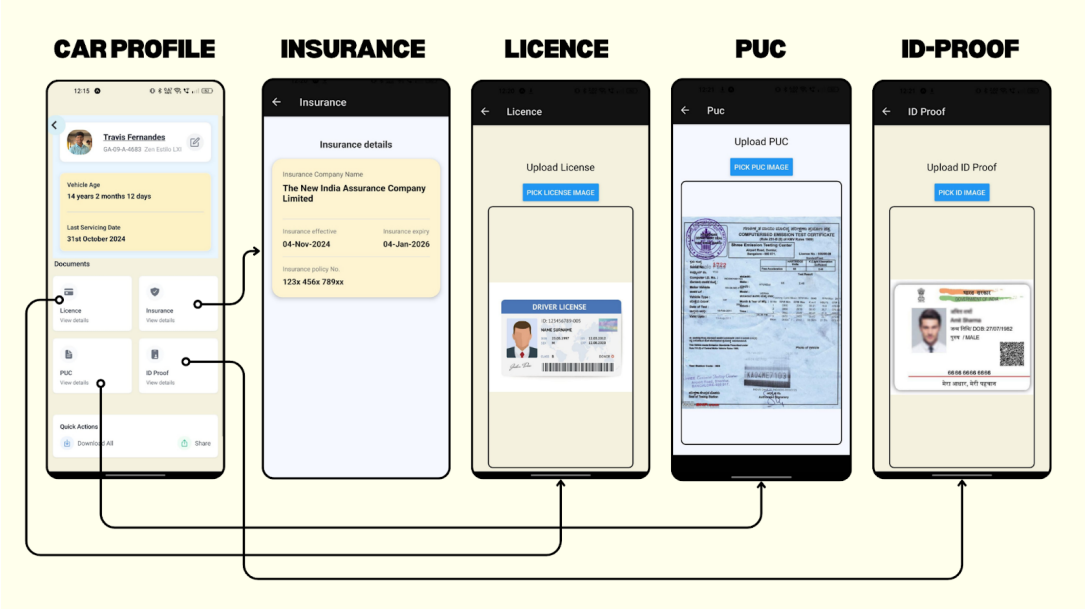

Fig. 5.10: Car Profile Overview Interface on Mobile Device

Figure 5.10 illustrates the Car Profile section of the ADAM mobile application, providing users with quick access to key vehicle details and document management features.

Insurance Details

Upon selecting the *Insurance* tile, users are navigated to a screen that displays:

- Insurance Company Name
- Effective and Expiry Dates of the insurance policy
- Policy Number, partially masked for security

This view allows users to verify insurance validity and renew it on time. The insurance data can either be fetched through APIs or entered/uploaded manually by the user.

Licence Upload

The *Licence* section enables users to upload a digital copy of their driver's license. It features:

- A simple interface with an Upload/Pick Image button
- A placeholder to preview the selected image

This ensures the license is stored digitally and can be accessed or verified during checks or renewals.

PUC (Pollution Under Control) Certificate

The *PUC module* facilitates uploading of the latest pollution control certificate. Features include:

- Image selection and preview interface
- Support for scanned documents or live photo capture

This ensures that emission compliance is met and documentation is always up to date.

ID Proof Management

This section is designed to store personal identity proof such as Aadhar Card or other government-issued identification. Key functionalities:

- Upload and preview interface
- Used for verification in processes like insurance claims, vehicle ownership, and servicing

Quick Actions

At the bottom of the Car Profile screen, two quick-access utilities are provided:

- **Download All:** Compiles all uploaded documents into a downloadable format (e.g., ZIP or PDF bundle).
- **Share:** Allows secure sharing of documents through email, messaging apps, or cloud services with appropriate privacy controls.

Location Feature

The **Location module** in the **ADAM mobile application** is designed to enhance user convenience by providing real-time discovery and navigation to essential automotive services. This feature leverages integrated mapping technologies to identify nearby facilities and offer intuitive route guidance.

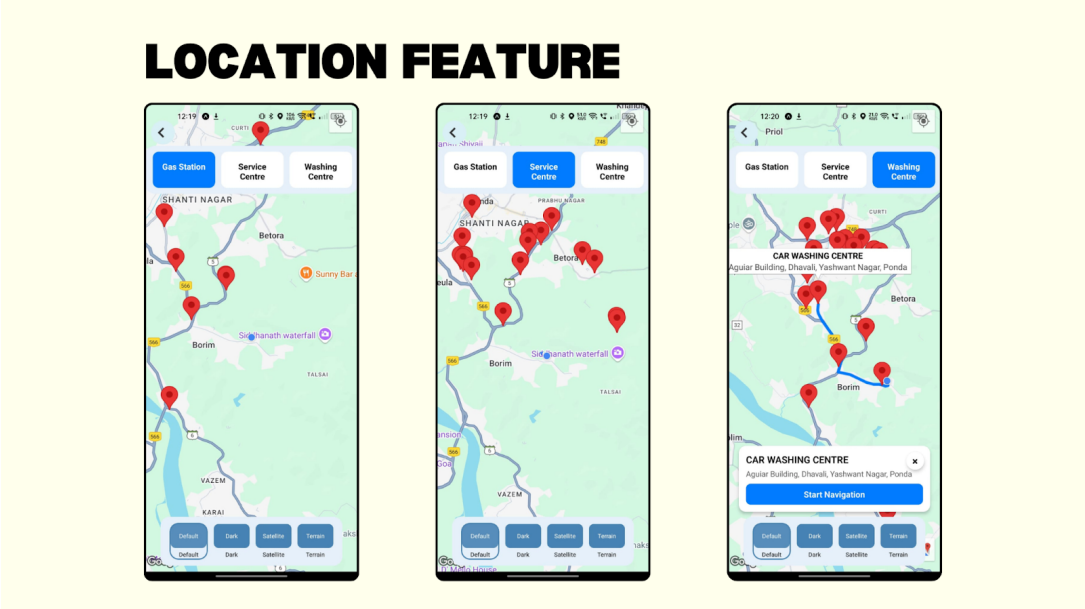

Fig. 5.11: Location Discovery and Navigation Interface on Mobile Device

Figure 5.11 illustrates the Location module interface of the ADAM mobile application, displaying nearby automotive services and route options.

Nearby Services Discovery The module allows users to locate key categories of automotive support services with a single tap:

- Gas Stations
- Service Centres
- Washing Centres

Each category is easily selectable via a toggle interface at the top of the screen. Once selected, the map dynamically updates to display the corresponding service providers as pins on the map, allowing users to visually explore the closest available options.

Interactive Map Experience Users can interact directly with the map to:

- Tap on pins to view brief details such as name and address.

- Retrieve precise location data for any selected service.
- Access a dedicated option to initiate turn-by-turn navigation directly from the application, ensuring seamless guidance to the chosen destination.

Map Themes and Views Recognizing user preferences and varying visibility needs, the Location module includes multiple map display themes:

- **Default Map View:** For standard daytime use.
- **Dark Mode:** For low-light environments, reducing eye strain.
- **Satellite View:** Offers a real-world visual overlay of terrain and structures.
- **Terrain Mode:** Provides an enhanced view of topographical features.

These options ensure the map interface remains versatile and user-friendly under different conditions.

Integrated Navigation On selecting a particular service location, the system displays detailed information along with a *Start Navigation* button. This directly launches guided navigation, providing optimized routes to the selected destination, thereby improving travel efficiency and reducing the likelihood of navigation errors.

CONCLUSION

The Automotive Diagnostic and Activity Monitoring (ADAM) System represents an innovative solution for improving vehicle safety, operational efficiency, and user convenience. The system combines real-time vehicle diagnostics, advanced data analysis, and seamless mobile application integration to provide actionable insights to various stakeholders, including vehicle owners, insurance agents, and regulatory authorities. By employing cutting-edge technologies such as machine learning algorithms, Raspberry Pi hardware, and the On-Board Diagnostics II (OBD-II) protocol, the ADAM System enables robust vehicle monitoring, fault detection, and forensic analysis capabilities.

The real-time diagnostic functionality of the ADAM System monitors vehicle performance and identifies potential mechanical issues before they escalate into significant problems, thereby reducing maintenance costs and enhancing road safety. The incorporation of machine learning facilitates predictive analytics, enabling the system to offer recommendations for preventive maintenance and optimize vehicle performance. Using Raspberry Pi as a cost-effective and scalable hardware platform allows for efficient data processing and ensures the adaptability of the system to a wide range of vehicles.

Integration with the OBD-II protocol ensures compatibility with modern vehicles and enables the collection of detailed diagnostic data, including engine performance, fuel efficiency, and emissions levels. The ADAM System also leverages a user-friendly mobile application to provide real-time notifications, comprehensive diagnostic reports, and driving insights, making it accessible and practical for everyday users.

Future advancements could expand the system's capabilities by incorporating features such as obstacle detection, which would enhance situational awareness for drivers. Predictive maintenance could be further refined to include more sophisticated failure predictions based on historical and real-time data. Additionally, integrating the system with advanced driver-assistance systems (ADAS) could contribute to autonomous driving technologies by providing critical diagnostic and environmental data. These developments would position the ADAM System as a pivotal technology in the evolution of intelligent and connected vehicles.

BIBLIOGRAPHY

1. Duy Le Nguyen, Myung-Eui Lee and A. Lensky, **"The design and implementation of new Vehicle Black Box using the OBD information,"** *2012 7th International Conference on Computing and Convergence Technology (ICCCT)*, Seoul, Korea (South), 2012, pp. 1281-1284.

2. P. R. Sawant and Y. B. Mane, **"Design and Development of On-Board Diagnostic (OBD) Device for Cars,"** *2018 Fourth International Conference on Computing Communication Control and Automation (ICCUBEA)*, Pune, India, 2018, pp. 1-4, https://doi.org/10.1109/ICCUBEA.2018.8697833.

3. R. Malekian, N. R. Moloisane, L. Nair, B. T. Maharaj and U. A. K. Chude-Okonkwo, **"Design and Implementation of a Wireless OBD II Fleet Management System,"** *IEEE Sensors Journal*, vol. 17, no. 4, pp. 1154-1164, Feb. 15, 2017, https://doi.org/10.1109/JSEN.2016.2631542.

4. Landry Frank Ineza Havugimana, Bolan Liu, Fanshuo Liu, Junwei Zhang, Ben Li and Peng Wan, **"Review of Artificial Intelligent Algorithms for Engine Performance, Control, and Diagnosis,"** *Energies*, vol. 16, 2023, pp. 1206, https://doi.org/10.3390/en16031206.

5. R. Ahmed, M. El Sayed, S. A. Gadsden, J. Tjong and S. Habibi, **"Automotive Internal-Combustion-Engine Fault Detection and Classification Using Artificial Neural Network Techniques,"** *IEEE Transactions on Vehicular Technology*, vol. 64, no. 1, pp. 21-33, Jan. 2015, https://doi.org/10.1109/TVT.2014.2317736.

6. J. B. Porch, C. Heng Foh, H. Farooq and A. Imran, **"Machine Learning Approach for Automatic Fault Detection and Diagnosis in Cellular Networks,"** *2020 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom)*, Odessa, Ukraine, 2020, pp. 1-5, https://doi.org/10.1109/BlackSeaCom48709.2020.9234962.

7. S. N. Narayanan, S. Mittal and A. Joshi, **"OBD_SecureAlert: An Anomaly Detection System for Vehicles,"** *2016 IEEE International Conference on Smart Computing (SMARTCOMP)*, St. Louis, MO, USA, 2016, pp. 1-6, https://doi.org/10.1109/SMARTCOMP.2016.7501710.

8. J. E. Meseguer, C. T. Calafate, J. C. Cano and P. Manzoni, **"Assessing the impact of driving behavior on instantaneous fuel consumption,"** *2015 12th Annual IEEE Consumer Communications and Networking Conference (CCNC)*, Las Vegas, NV, USA, 2015, pp. 443-448, https://doi.org/10.1109/CCNC.2015.7158016.

9. R. Massoud, F. Bellotti, R. Berta, A. De Gloria and S. Poslad, **"Exploring Fuzzy Logic and Random Forest for Car Drivers' Fuel Consumption Estimation in IoT-Enabled Serious Games,"** *2019 IEEE 14th International Symposium on Autonomous Decentralized System (ISADS)*, Utrecht, Netherlands, 2019, pp. 1-7, https://doi.org/10.1109/ISADS45777.2019.9155706.

10. T. Liu, G. Yang and D. Shi, **"Construction of Driving Behavior Scoring Model based on OBD Terminal Data Analysis,"** *2020 5th International Conference on Information Science, Computer Technology and Transportation (ISCTT)*, Shenyang, China, 2020, pp. 24-27, https://doi.org/10.1109/ISCTT51595.2020.00012.